



A Transfer-Learned Graph Neural Network and Blockchain-Based Framework for Temporal Fraud Detection: From Countering Temporal-Echo Topology Poisoning Attacks in SDVNs to Tax Evasion Analysis

An undergraduate project progress report submitted to the
Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfillment of the requirements for the

**Degree of the Bachelor of the Science of Engineering
Honours**

by

Kumara K.N.Y.	- EG/2021/4624
Bandara A.W.M.L.M.	- EG/2021/4426
Wijegurusinghe O.W.D.K.M	- EG/2021/4890
Hewawasam A.D.K.	- EG/2021/4556

.....
Dr. Nilmantha Wijesekara
(Supervisor)

.....
Dr. Prabath Weerasingha
(Co-Supervisor)

Abstract

Software-Defined Vehicular Networks (SDVNs) are increasingly vulnerable to sophisticated Temporal-Echo Topology Attacks that manipulate timestamps, inject fake heartbeat signals, and create phantom network links. Current defense mechanisms rely on rule-based detection or basic machine learning approaches, leaving critical security gaps. This project proposes a novel integrated defense framework combining Graph Neural Networks (GNNs) for real-time anomaly detection with blockchain-based verification for tamper-proof audit trails. The solution specifically targets three attack variants: (1) Topology Time-Warp (timestamp manipulation), (2) Beacon State Heartbeat Hijack Attack (fake control messages), and (3) Multipath Echo (phantom links). We will implement the system using NS-3 network simulation, TensorFlow for GNN modeling, and Hyperledger Fabric for blockchain integration. Furthermore, we will apply transfer learning to demonstrate the framework's generalization to financial fraud detection (tax evasion), establishing a unified temporal fraud detection system. This high-novelty research addresses previously unstudied attack-defense combinations and is intended for publication in top-tier IEEE/ACM journals.

Keywords : Software-Defined Vehicular Networks, Graph Neural Networks, Blockchain, Topology Attacks, Anomaly Detection, Transfer Learning, Cybersecurity

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Dr. ■ Wijesekara, for his invaluable guidance, continuous support, and encouragement throughout the development of this undergraduate project titled “*A Transfer-Learned Graph Neural Network and Blockchain-Based Framework for Temporal Fraud Detection: From Countering Temporal-Echo Topology Poisoning Attacks in SDVNs to Tax Evasion Analysis.*” His expertise, insightful feedback, and dedication have been instrumental in shaping the direction and success of this work. We also extend our heartfelt thanks to our co-supervisor, Dr. ■ Weerasingha, for his constructive suggestions, guidance, and support in refining this proposal. His contributions have greatly enhanced the quality of our research. Furthermore, we would like to acknowledge all the academic staff and coordinators involved in the undergraduate project module for their guidance and support throughout this journey. Finally, we express our sincere appreciation to everyone who has contributed directly or indirectly to the successful completion of this project. Their encouragement and support have been truly valuable.

Contents

Acronyms	viii
1 Introduction	1
1.1 Background	1
1.1.1 Control Plane Protocols and Timing Dependence	1
1.2 Problem Introduction	2
1.3 Limitations of Existing Approaches	3
1.4 Problem Statement	3
1.5 Research Objectives	3
1.5.1 General Objective	3
1.5.2 Specific Objectives	4
1.6 Research Scope	4
1.7 Motivations	4
1.8 Methodological Contributions	5
1.9 Novelty Statement	5
1.10 Implications of the Study	6
2 Background and Related Work	7
2.1 Background: Temporal Attacks in SDN and SDVN	7
2.1.1 Phantom Link and Time-Delay Attacks	7
2.1.2 Timing Side-Channels and Reconnaissance	8
2.1.3 Detection Challenges	8
2.1.4 Critical Implications for SDVN	8
2.2 Related Work and Research Gaps	9
2.2.1 Gap 1: Absence of Integrated Detection for SDVN-Specific Temporal Attacks	9
2.2.2 Gap 2: Lack of Trustworthy Verification and Automated Response	9
2.2.3 Gap 3: Domain-Specific Silos in Temporal Fraud Detection . .	9
2.2.4 Gap 4: Inadequate Comparative Evaluation Frameworks . .	10
2.2.5 Related Work Comparison	10
3 Methodology	12
3.1 Dual Mode framework	12
3.2 System Architecture	16
3.2.1 Data Plane Layer	17
3.2.2 Control and Intelligence Layers	18
3.2.3 Verification and Application Layers	18
3.3 High-Level Functional Workflow	18
3.4 Formulation of Temporal-Echo Attacks in SDVNs	19

3.4.1	Topology Time Warp (TTW) Attack	19
3.4.2	Beacon State Heartbeat Hijack Attack (BSHH) Attack . . .	28
3.4.3	Multipath Echo (ME) Attack	36
3.4.4	Attack Signature Characterization	44
3.4.5	Detection Logic	49
3.4.6	Cryptographic Placement Analysis	52
3.4.7	Mitigation Algorithms	54
3.5	Attack Impact Framework	57
3.6	Cross-Domain Temporal Pattern Analysis	58
4	Timeline and Resource Required	62
4.0.1	Baseline Selection and Benchmarking Rationale	62
4.0.2	Baseline Selection and Benchmarking Rationale	62
4.1	Timeline	64
4.2	Resource Required	64
4.2.1	Hardware Resources	64
4.2.2	Software Resources (All Open-Source/Free)	65
4.2.3	Budget Summary	65
5	Conclusion	66
	References	67

List of Figures

3.1	Proposed Layered System Architecture for Secure Temporal Anomaly Detection in SDVN	17
3.2	Proposed Attack Detection and Mitigation Framework Workflow . .	19
3.3	TTW Attack via Malicious Vehicle (Without RSU): V1 manipulates timestamps of HELLO messages sent to the controller.	20
3.4	TTW Attack via Malicious RSU: RSU manipulates timestamps in forwarded vehicle HELLO messages.	22
3.5	TTW Attack via Malicious Controller (Without RSU): The controller internally manipulates timestamps of received HELLO messages.	24
3.6	TTW Attack via Malicious Controller (With RSU): The controller internally manipulates timestamps of received HELLO messages. . .	26
3.7	BSHH Attack via Malicious Vehicle (Without RSU): V2 hijacks heartbeat messages from V3 to V1, inducing false adjacency. . . .	28
3.8	BSHH Attack via Malicious RSU: The RSU manipulates the sender identity of heartbeat messages to induce false adjacency.	30
3.9	BSHH Attack via Malicious Controller (Without RSU): The controller internally hijacks heartbeat identities to induce false adjacency. .	32
3.10	BSHH Attack via Malicious Controller (With RSU): The controller internally hijacks heartbeat identities to induce false adjacency. . .	34
3.11	Vehicle-based Multipath Echo Attack (Without RSU): V3 and V4 replay V1–V2 topology to the controller, creating false paths. . . .	36
3.12	The RSU injects duplicated topology information related to V1–V2, creating false paths. (With RSU)	38
3.13	Controller-based Multipath Echo Attack (Without RSU): Controller internally duplicates V1–V2 link across multiple inferred paths. . .	40
3.14	Controller-based Multipath Echo Attack (With RSU): The controller internally duplicates the V1–V2 link across multiple inferred paths.	42
4.1	Project Timeline (Gantt Chart)	64

List of Tables

2.1	Comparative Analysis of Related Work: Coverage of Temporal-Echo Topology Poisoning Attacks in SDVNs. Y = Yes; P = Partial; N = No; NP = None Proposed; – = Not Applicable. Columns G1–G4 indicate coverage of the four identified research gaps (G1: All three attack variants addressed; G2: Dual-mode LW + FS versions provided; G3: Formal attack signatures derived; G4: Cryptographic role analyzed per attacker position).	11
3.1	Detection Metrics [19]	57
4.1	Hardware Resource Requirements	64
4.2	Software Resource Requirements	65
4.3	Overall Project Budget	65

Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
ARP	Address Resolution Protocol
AUROC	Area Under the Receiver Operating Characteristic
BSHH	Beacon State Heartbeat Hijack
DDoS	Distributed Denial of Service
DoS	Denial of Service
FN	False Negative
FP	False Positive
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GNN	Graph Neural Network
IDS	Intrusion Detection System
LLDP	Link Layer Discovery Protocol
ME	Multipath Echo
ML	Machine Learning
NS-3	Network Simulator Version 3
OBU	On-Board Unit
PBFT	Practical Byzantine Fault Tolerance
PKI	Public Key Infrastructure
QoS	Quality of Service
RSU	Roadside Unit
SDN	Software-Defined Networking

SDVN Software-Defined Vehicular Network

SUMO Simulation of Urban Mobility

TGN Temporal Graph Network

TN True Negative

TP True Positive

TTW Topology Time-Warp

VANET Vehicular Ad-hoc Network

Chapter 1

Introduction

1.1 Background

The rapid development of intelligent transportation systems and autonomous driving technologies depends heavily on reliable, low-latency, and adaptive communication networks. Vehicular Ad-hoc Networks (VANETs) enable direct communication among vehicles and roadside units, but their highly dynamic topology makes traditional distributed routing and management inefficient [25, 26]. Software-Defined Vehicular Networks (SDVNs) integrate the principles of Software-Defined Networking (SDN) with vehicular networking, enabling centralized control, programmability, and global visibility of the network state [25, 27].

In SDVN architecture, a centralized controller maintains a global view of vehicles, roadside units, and communication links, and dynamically manages routing, traffic engineering, and network policies [18, 27]. The separation of control and data planes allows rapid adaptation to mobility, traffic density changes, and application requirements. These capabilities make SDVN a promising foundation for safety-critical services such as collision avoidance, cooperative driving, and emergency response systems [26, 27].

However, this architectural shift also changes the security model. Traditional vehicular networks relied on distributed decision-making, whereas SDVN concentrates control intelligence in a central controller. The correctness of the entire network depends on the accuracy and timeliness of topology and status information reported to the controller. Any manipulation of this control-plane information can lead to incorrect routing, service disruption, and potentially dangerous decisions affecting vehicle safety [18, 24, 27].

1.1.1 Control Plane Protocols and Timing Dependence

The SDVN control plane relies on a set of standardized communication protocols to support centralized monitoring and network control [25]. The southbound interface between the controller and RSUs is implemented using the OpenFlow protocol (version 1.5) over TCP port 6653 [25]. Wireless communication between vehicles and RSUs is supported through IEEE 802.11p and Wireless Short Message Protocol (WSMP), with periodic beaconing used to convey mobility and link-state

information.

Topology discovery in the considered system model is performed through the aggregation of beacon messages periodically broadcast by vehicles, rather than through LLDP-based link-layer discovery. Lightweight beacon messages are transmitted at fixed intervals to report node presence, neighborhood information, and link conditions to RSUs, which forward aggregated topology state updates to the controller [20, 26]. In addition, heartbeat-style beacon mechanisms are employed to maintain network liveness, with small control messages transmitted at regular intervals.

To secure control-plane communications, Transport Layer Security (TLS 1.3) is assumed to provide cryptographic protection against eavesdropping and message tampering [7]. Despite these protections, the control plane remains highly dependent on correct timing, sequencing, and message freshness. As a result, SDVN control mechanisms are inherently vulnerable to attacks that exploit temporal inconsistencies, delayed or replayed messages, and out-of-context reuse of valid control information [20, 23]

1.2 Problem Introduction

One of the most serious threats to SDVNs is topology poisoning, where adversaries manipulate topology discovery or status update mechanisms to mislead the controller [18, 24, 27].

While many existing studies focus mainly on **structural attacks**—such as fake nodes, fake links, or forged connectivity—recent research has shown that attackers can also exploit the **temporal properties** of control-plane messages [20, 21]. Instead of changing who is connected to whom, these attacks manipulate *when* events appear to have happened.

Such time-based manipulations create inconsistencies between the real network and the controller’s perceived network state. These attacks are often subtle, low-volume, and difficult to detect using conventional rule-based or volume-based security mechanisms [19, 20].

In this research, three novel variants of topology poisoning attacks are identified:

- **Topology Time-Warp (TTW) Attack** – temporal manipulation through timestamp or sequence forgery of stale topology updates
- **Beacon State Heartbeat Hijack (BSHH) Attack** – temporal deception via heartbeat identity spoofing during forwarding
- **Multipath Echo (ME) Attack** – **echo-based spatial attack** creating phantom multipath topology through legitimate link duplication

Attack Classification:

The first two attacks (TTW, BSHH) distort the controller’s perception of time by reusing valid information with incorrect temporal context. The third attack (ME) represents a special form of spatial topology poisoning not explicitly studied

in SDVN literature [19, 24]. It replays legitimate topology information across multiple reporting paths, creating a false spatial network structure without primarily manipulating timestamps [24].

These attacks evade detection by reusing valid control-plane data in the wrong temporal or spatial context, exploiting the controller’s assumption of honest, fresh, and location-consistent reporting [19, 20]. This creates dangerous inconsistencies between physical reality and the controller’s network perception, potentially compromising safety-critical vehicular services [27].

1.3 Limitations of Existing Approaches

Most existing SDN and SDVN security mechanisms are designed to detect structural anomalies such as fake links, unknown nodes, or abnormal traffic volumes. Techniques such as rule-based validation (e.g., TopoGuard), active probing, and shallow machine learning models are effective against overt topology poisoning and denial-of-service attacks [18, 24, 28]. However, they suffer from several limitations:

- They primarily analyze static structure rather than time-evolving behavior
- They assume attacks introduce clearly invalid or inconsistent data
- They rely on predefined rules or shallow features that cannot capture subtle temporal deception
- They provide limited support for forensic analysis and accountability after an attack occurs

As a result, attacks that reuse valid data but manipulate timing, ordering, or repetition can bypass existing defenses [19–21]. Furthermore, current solutions are usually domain-specific and do not consider whether similar temporal deception patterns appear in other domains such as financial fraud or tax evasion [11, 12].

1.4 Problem Statement

In safety-critical vehicular environments where SDVNs rely on timely control-plane updates under high mobility and strict latency constraints, existing security mechanisms—designed mainly for structural and volume-based anomalies—fail to detect temporal-echo topology attacks that manipulate the timing, ordering, or replay of valid control messages, leading to incorrect controller views and unsafe routing decisions [18, 20, 24, 27].

1.5 Research Objectives

1.5.1 General Objective

To study and formally characterize temporal-echo topology attacks in Software-Defined Vehicular Networks (SDVNs) and analyze their evasion of existing security mechanisms.

1.5.2 Specific Objectives

1. To formally define three temporal-echo attack variants (TTW, BSHH, ME) and their manipulation mechanisms.
2. To analyze why existing SDVN security mechanisms fail to detect these attacks.
3. To explore cross-domain similarities between temporal deception patterns in SDVNs and financial fraud scenarios [6, 11, 12].

1.6 Research Scope

This study is limited to conceptual, theoretical, and analytical investigation, without implementation or experimental validation. The scope of the research is defined as follows:

The work focuses only on the SDVN control plane, and does not consider data-plane forwarding performance, physical-layer behavior, or vehicle mobility modeling. The study is restricted to formal modeling, threat characterization, and analytical comparison based on existing literature and protocol behavior.

No system implementation, simulation, or prototype development is carried out. In particular, the study does not include NS-3 simulations, GNN model training, blockchain deployment, real-world experiments, or hardware-based evaluation. All findings are derived from conceptual modeling, mathematical formalization, and comparative analysis of existing mechanisms.

The temporal focus of the study is limited to analyzing how topology information is manipulated through timing, ordering, and replay, and does not address other attack categories such as jamming, physical tampering, or cryptographic compromise.

Future phases of this research may extend to implementation and experimental validation, but such activities are explicitly outside the scope of the present study.

1.7 Motivations

We pursue this research driven by three key factors.

Security of Future Transportation Systems: With autonomous and connected vehicles proliferating, network control failures pose direct risks to human lives. Subtle attacks on control timing stand out as particularly insidious—they evade detection and can linger undetected for extended durations.[19, 20].

Inadequacy of Current Defenses: Today’s SDVN protections target blatant topology changes or traffic floods, leaving stealthy temporal deceptions unaddressed . [18, 24].

Cross-Domain Relevance of Temporal Fraud: Time-based tricks extend beyond networks. In finance, they underpin fraud, tax dodging, and money laundering through delays, reordering, or replays of valid events.[3, 6, 11, 12].

1.8 Methodological Contributions

This work delivers four core methodological advances:

1. Temporal Properties Framework for Topology Analysis We introduce a framework that breaks down topology attacks by temporal traits—timestamp validity, sequence integrity, and repetition patterns—instead of conventional structural ones like node or link checks. It uncovers time-driven deceptions that slip past structural safeguards. [18].

2. Temporal-Echo Attack Taxonomy We propose a fresh taxonomy for temporal-echo topology attacks, rooted in three primitives: time reordering, heartbeat spoofing, and phantom duplication. These reuse valid control messages yet shatter temporal consistency, forcing controller state drift.

3. Graph-Based Cross-Domain Pattern Matching We devise a method to map SDVN temporal attacks onto financial fraud via temporal adjacency matrices. It reveals matching anomaly patterns between Topology Time-Warp attacks and transaction replay schemes. [6, 11].

4. Temporal Evasion Feature Extraction We pinpoint evasion-enabling temporal signals—inter-packet delays, sequence gaps, repetition cycles—that dodge rule-based and volume detectors. These offer precise hooks for upcoming GNN detection tools.

1.9 Novelty Statement

This study claims novelty through two fundamental contributions:

1. “Temporal-Echo Attacks” as a New Attack Class Temporal-echo attacks represent a previously unclassified family of SDVN topology attacks that reuse legitimate control messages through manipulated timing rather than fabricating false topology data. Unlike structural topology poisoning (fake nodes/links [18]), temporal-echo attacks preserve structural validity while violating:

- **Temporal:** timestamp manipulation creates time-warped topology views,
- **Echo:** legitimate messages are replayed or duplicated under altered timing contexts.

Novel modeling: Formalized as (G_t, T, E) where G_t is the topology graph at time t , T denotes manipulated timestamps, and E represents echo operations (replay/duplication). Existing taxonomies do not explicitly model this class [20].

2. First Cross-Domain Temporal Fraud Mapping Presents the first graph-theoretic analysis linking SDVN temporal deception to financial fraud patterns:

- TTW attack $\leftrightarrow$ transaction replay (temporal reordering) [6],
- BSHH attack $\leftrightarrow$ liveness spoofing in financial systems [11],
- ME attack $\leftrightarrow$ shell-company networks (phantom duplication).

Novel modeling: Temporal adjacency matrices reveal isomorphic anomaly structures across domains, enabling principled transfer learning opportunities.

1.10 Implications of the Study

This study yields key implications across theoretical, practical, policy, and future research domains. **Theoretical Implications.** It advances network security theory by positioning temporal deception as a core attack primitive, on par with structural and volume threats. New taxonomies must now elevate time to a primary security attribute . [18, 20]. Formalizing temporal-echo attacks as (Gt, T, E) equips researchers with a solid lens for dissecting timing manipulations in networked control systems.

Practical Implications. The results spotlight vulnerabilities in SDVN control planes that underpin vital functions like collision avoidance and platooning. Temporal attacks maintain structural validity yet trigger flawed routing and forwarding—stealthy risks to smart transportation that elude current detectors . [19, 27]. Operators must rethink topology validation and monitoring in live SDVN setups.

Policy Implications. Parallels between SDVN temporal attacks and financial fraud call [6, 11] for unified cross-sector standards on temporal integrity. Regulators and standards groups need to mandate temporal checks in safety-critical protocols, mirroring today’s structural and crypto requirements.

Future Research Implications. The work lays groundwork for adaptable temporal anomaly detection. Isomorphisms to financial fraud open doors to transfer learning across fields, while evasion features like inter-packet delays and sequence gaps guide GNN detector design. Above all, it shifts validation focus: trust demands verifying not just what arrives, but when and in what order—reshaping security for tomorrow’s networks.

Chapter 2

Background and Related Work

2.1 Background: Temporal Attacks in SDN and SDVN

While no research specifically addresses “Temporal-Echo Attacks,” related temporal manipulation techniques have been identified in Software-Defined Networks (SDN) and Software-Defined Vehicular Networks (SDVN) that fundamentally differ from traditional structural attacks [1, 2, 26, 28]. Unlike topology poisoning that creates fake links or nodes, these temporal attacks manipulate the timing, sequencing, and ordering of legitimate control-plane messages. This corrupts the controller’s temporal perception of network events rather than its structural understanding [18, 19].

The attack vector lies in *when* events are perceived, rather than in the content of packets, creating a uniquely challenging detection problem [1, 4, 26].

2.1.1 Phantom Link and Time-Delay Attacks

Phantom Link Attacks: Weintraub et al. (2024) demonstrated that attackers can exploit transient synchronization windows during flow rule installation to create unauthorized connectivity within brief 50+ millisecond windows, leaving no trace once the network converges [20].

Time-Delay Attacks: Ramani et al. (2022) showed that intercepting and delaying Link Layer Discovery Protocol (LLDP) packets by 10–50 milliseconds causes topology convergence failures [21]. In SDVNs, where high vehicle mobility creates dynamic topologies, even minor delays can cause incorrect routing and compromise safety-critical communications [27].

2.1.2 Timing Side-Channels and Reconnaissance

Shoaib et al. (2021) showed that measuring flow rule response times allows adversaries to infer recent communications, cache states, and potentially vehicle proximity [22].

Liu et al. (2017) demonstrated that timing analysis combined with active probing enables reconstruction of network topology and communication patterns [23]. These studies reveal that timing itself is an information channel, exploitable for both passive reconnaissance and active temporal manipulation [24, 28].

2.1.3 Detection Challenges

Temporal attacks evade conventional defenses due to several inherent characteristics:

- **Stealth through legitimacy:** Individual messages are factually correct; deception lies in their temporal arrangement [18].
- **Minimal operational footprint:** They succeed with low traffic volume, bypassing volumetric anomaly detection [19, 26].
- **Exploitation of transient states:** They operate within legitimate windows of state inconsistency during topology updates [1, 4].
- **Evasion of static patterns:** Lack of distinctive signatures for pattern-matching detection systems [8, 9, 20].
- **Requirement for temporal reasoning:** Detection demands understanding of event causality and sequence consistency [5, 6, 18].

2.1.4 Critical Implications for SDVN

These attacks are particularly concerning in vehicular contexts. Safety applications such as collision avoidance demand latencies under 100 milliseconds [1, 2]. Temporal attacks introducing 50+ millisecond delays or misordered events can directly compromise these guarantees [4, 27]. Frequent topology changes due to vehicle mobility expand the attack surface of exploitable temporal windows, making this a pressing concern for intelligent transportation security [2, 3, 26, 28].

2.2 Related Work and Research Gaps

2.2.1 Gap 1: Absence of Integrated Detection for SDVN-Specific Temporal Attacks

Current State: Existing SDN detection mechanisms—rule-based systems (TopoGuard [24]), traditional ML (SecTopo [19]), or hybrid approaches—focus on structural attacks and cannot effectively model temporal consistency or event causality [22, 23]. They are rarely evaluated under SDVN constraints like high mobility and strict latency requirements [2, 21].

Research Opportunity: Systematic temporal attack characterization in SDVN contexts remains unexplored [18, 27].

2.2.2 Gap 2: Lack of Trustworthy Verification and Automated Response

Current State: Blockchain use in SDVN is largely passive, primarily for auditing or trust management [7, 26]. Verification and response typically require human intervention.

Research Opportunity: Active blockchain integration for temporal attack verification represents future work [20, 28].

2.2.3 Gap 3: Domain-Specific Silos in Temporal Fraud Detection

Current State: Graph Neural Network (GNN) models for anomaly detection exist separately in network security [1, 13] and financial fraud detection [3, 5, 11], but no study explores whether temporal fraud patterns learned in one domain are transferable to another [3, 5, 18].

Research Opportunity: Cross-domain temporal pattern transfer analysis remains an open research question [1, 11, 18].

2.2.4 Gap 4: Inadequate Comparative Evaluation Frameworks

Current State: SDVN security studies often lack benchmarking across multiple baselines (rule-based, ML, static graph learning, blockchain-only) [19, 24, 26], limiting the assessment of new approaches.

Research Opportunity: Structured comparative evaluation frameworks for temporal attack detection need development [21, 27].

2.2.5 Related Work Comparison

Table ?? provides a comparative summary of existing defense mechanisms and their limitations in detecting temporal attacks in SDN and SDVN environments.

6	Jain et al. [?] (Sec-Topo)	2024	LLDP poisoning	topology	Hybrid Autoencoder + Decision Tree in P4/Tofino data plane; 98.76% accuracy	In-network P4 pipeline blocking	N	N	N	N	N	N	N	LLDP-specific features in-applicable to beacon-based SDV/N; no temporal ordering or heartbeat identity modeling
7	Adjou et al. [?] (TopoTrust)	2022	HTS host tracking; OFDP link discovery poisoning	Blockchain authenticity verification	Smart contract isolation (96% fast-relocation; 100% link fabrication)	N	N	N	N	N	N	N	N	Verifies structural message authenticity only; structurally valid replayed beacons (TTW/BSHH) pass blockchain verification without detection
8	Marionette [?] (CCS 2024)	2024	Globalized topology poisoning via control-plane flow forwarding (CVE-2024-37018)	NP	NP	N	N	N	N	N	N	N	N	Exposes new structural attack via control privileges; no detection mechanism proposed; does not address temporal message manipulation
9	TopoSleuth [?]	2025	Link fabrication (forgery, relay, replay); topology freezing; switch spoofing; LLDP flooding	Decoy trip-wires + behavioral profiling + multi-hop validation (4-module framework)	Correlation-based response via Topology Monitor	P	N	N	N	N	N	N	N	Decoy mechanism targets LLDP packet replay; cannot detect beacon-based TTW, heartbeat identity BSHH, or multi-path echo ME in vehicular control plane
10	Deng et al. [?]	2024	Five structural topology poisoning threats exploiting VANET high mobility	Empirical countermeasures discussed but not implemented	Countermeasure discussion only	N	Y	P	N	N	N	N	N	Closest SDV/N work; targets structurally invalid topology data only; temporal-echo attacks that reuse valid messages in wrong context fall outside this threat model
11	Jayakrishna & Pranth [?]	2025	DDoS attacks in VANETs	Hybrid CNN + LSTM	Traffic filtering	P	P	N	N	N	N	N	N	Volumetric DDoS detection via rate/flow features; inapplicable to low-volume control-plane temporal consistency attacks (TTW/BSHH/ME)
12	Mekonen et al. [?]	2025	Position falsification in VANET BSMs	Ensemble (KNN + Bagging, RF, DT, MLP); 100% F1 (Attack 1); 97.85% (Attack 16)	ML RSU alert broadcast	N	N	P	N	N	N	N	N	BSM payload features (position, speed, direction) are unrelated to SDN control-plane temporal features (timestamp ordering, reporter multiplicity)
13	Tariq [?]	2024	BeiDou spoofing in VANET	GNSS Autoencoder + LSTM	Distributed encryption	P	N	N	N	N	N	N	N	GNSS signal features (Doppler, carrier phase) are unrelated to SDN beacon timestamp and heartbeat identity features; no lightweight version; no SDV/N context
14	Yang et al. [?] (GNN-NAD)	2025	DDoS; web-based attacks in SDN consumer electronics	GraphSAGE + RF	Static vulnerability graph fused with dynamic traffic	P	N	N	N	N	N	N	N	Static attack graph cannot model continuously evolving SDV/N topology; no temporal snapshot sequence; no vehicular mobility or beacon-interval

Chapter 3

Methodology

3.1 Dual Mode framework

This work proposes a dual-mode framework. The Lightweight Version (LW) employs rule-based attack signature detection and HMAC-timestamp-nonce verification, targeting resource-constrained OBU and RSU deployment. The Full-Stack Version (FS) employs Temporal Graph Network-based detection combined with threshold/aggregate signature-based authentication using Kyber+Saber key agreement and Hyperledger Fabric smart contract mitigation. Both versions are evaluated independently and comparatively. This design explicitly addresses deployment feasibility concerns [C4] while providing maximum detection performance when computational resources permit [B5, C1].

The temporal-echo attack class is formally modeled as the triple $\mathcal{A} = (G_t, \mathbf{T}, \mathbf{E})$, where $G_t = (\mathcal{V}_t, \mathcal{E}_t)$ is the true topology graph at time t , $\mathbf{T} \in R^{|\mathcal{V}| \times |\mathcal{V}|}$ is the timestamp manipulation matrix whose entry $\mathbf{T}_{ij}$ denotes the forged timestamp injected on the report of link e_{ij} , and $\mathbf{E} \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{V}|}$ is the echo operation matrix where $\mathbf{E}_{ij} = 1$ indicates that link e_{ij} has been replayed or duplicated by at least one unauthorized reporter. The resulting divergence between the controller-perceived topology G_t^C and the real topology G_t^R is quantified as:

$$\delta(G_t^C, G_t^R) = |\mathcal{E}_t^C \triangle \mathcal{E}_t^R| > 0 \quad (3.1)$$

where $\triangle$ denotes the symmetric difference operator and $\mathcal{E}_t^C, \mathcal{E}_t^R$ are the edge sets of the controller-perceived and real topology graphs, respectively. A non-zero divergence indicates a successful temporal-echo attack.

Let $\tau_r^{(k)}$ denote the controller reception time of the k -th beacon, $\tau_s^{(k)}$ the embedded sender timestamp, T_b the nominal beacon interval (100 ms per IEEE 802.11p), and ϵ the maximum tolerable one-way propagation delay.

TTW-S1 (Timestamp Staleness): A received beacon whose reception-timestamp offset exceeds one beacon interval plus propagation tolerance is classified as stale:

$$\Delta_k = \tau_r^{(k)} - \tau_s^{(k)} > T_b + \epsilon \quad (3.2)$$

TTW-S2 (Sequence Inversion): A later-received beacon carries an earlier sender timestamp than a previously received beacon from the same source, indicating replay of an older message:

$$\exists j < k : \tau_s^{(j)} > \tau_s^{(k)} \wedge \tau_r^{(j)} < \tau_r^{(k)} \quad (3.3)$$

TTW-S3 (Cross-Reporter Inconsistency): Two distinct reporters of the same link e_{ij} carry timestamps differing by more than one beacon interval, indicating at least one replayed report:

$$\exists V_a \neq V_b : \left| \tau_s^{V_a \rightarrow C}(e_{ij}) - \tau_s^{V_b \rightarrow C}(e_{ij}) \right| > T_b \quad (3.4)$$

BSHH-S1 (Duplicate Liveness Assertion): Two distinct physical senders submit heartbeats claiming to originate from the same node V_k within the same observation window:

$$\exists V_a \neq V_b : \text{Sender}(H^{V_a}) = \text{Sender}(H^{V_b}) = V_k, \quad \tau_r^{V_a} \approx \tau_r^{V_b} \quad (3.5)$$

where H^{V_a} denotes the heartbeat message received from physical sender V_a .

BSHH-S2 (Timestamp Regression): The heartbeat timestamp of node V_i decreases between consecutive controller-received messages, indicating replay of an older heartbeat:

$$\tau_s^{(k+1)}(V_i) < \tau_s^{(k)}(V_i) \quad (3.6)$$

BSHH-S3 (Liveness-Beacon Inconsistency): A heartbeat asserts liveness of V_i during window W in which no beacon was received from V_i , indicating identity spoofing:

$$\text{Heartbeat}(V_i, t) = \text{TRUE} \wedge \text{Beacon}(V_i, [t - W, t]) \quad (3.7)$$

ME-S1 (Reporter Count Excess): The number of distinct reporters $|\mathcal{R}(e_{ij}, t)|$ of link e_{ij} at time t exceeds the maximum physically plausible count given vehicle density λ and communication range r_{comm} :

$$|\mathcal{R}(e_{ij}, t)| > \rho_{\text{max}}(\lambda, r_{\text{comm}}) \quad (3.8)$$

where $\rho_{\text{max}}(\lambda, r_{\text{comm}}) = \lfloor \lambda \cdot \pi r_{\text{comm}}^2 \rfloor$ is the expected maximum number of vehicles within communication range under density λ .

ME-S2 (Path Diversity Inflation): The number of controller-inferred paths between V_i and V_j increases by more than the mobility-consistent bound δ_{max} in one beacon interval:

$$|\mathcal{P}(V_i, V_j)|_t - |\mathcal{P}(V_i, V_j)|_{t-1} > \delta_{\text{max}} \quad (3.9)$$

ME-S3 (Reporter-Range and Signal Inconsistency): Reporter V_k claims to observe link e_{ij} but either GPS-attested position places V_k beyond communication range of both endpoints, or the reported received signal strength is inconsistent with claimed proximity:

$$V_k \in \mathcal{R}(e_{ij}, t) \wedge \left(d(\text{pos}_{V_k}, e_{ij}) > r_{\text{comm}} \vee \text{RSSI}_{V_k \leftarrow V_i} < \text{RSSI}_{\text{min}}(r_{\text{comm}}) \right) \quad (3.10)$$

where $d(\text{pos}_{V_k}, e_{ij})$ denotes the Euclidean distance from the GPS-attested position of V_k to the nearest endpoint of link e_{ij} , and $\text{RSSI}_{\min}(r_{\text{comm}})$ is the minimum expected received signal strength at maximum communication range under the path-loss model.

For each incoming topology event e , the lightweight detector evaluates all nine signatures over the sliding window $\mathcal{W}$ and computes a weighted match score:

$$s(e) = \sum_{k=1}^9 w_k \cdot \mathbf{1}[\text{Sig}_k \text{ triggered}] \quad (3.11)$$

where $w_k \geq 0$ is the weight assigned to signature Sig_k , $\sum_{k=1}^9 w_k = 1$, and $\mathbf{1}[\cdot]$ is the indicator function. A detection alert is raised when $s(e) > \theta_{\text{LW}}$, where θ_{LW} is optimized via ROC analysis on simulation data.

The per-event time complexity of the lightweight detector is:

$$T_{\text{LW}}(|\mathcal{W}|, |\mathcal{S}|) = O(|\mathcal{W}| \cdot |\mathcal{S}|) \quad (3.12)$$

Since $|\mathcal{S}| = 9$ is a constant determined by the attack model, this reduces to:

$$T_{\text{LW}}(|\mathcal{W}|) = O(|\mathcal{W}|) \quad (3.13)$$

where $|\mathcal{W}|$ is the sliding window size. The detection overhead therefore scales linearly with the number of retained events, confirming deployability on resource-constrained OBUs and RSUs.

Each beacon or topology update message m from node V_i at sender timestamp τ_s is authenticated as:

$$m' = m \parallel \tau_s \parallel \text{nonce}_i \quad (3.14)$$

$$\text{MAC}_i = \text{HMAC-SHA256}(K_{V_i, \text{RSU}}, m') \quad (3.15)$$

The verifier accepts the message if and only if all three conditions hold simultaneously:

$$\text{HMAC-SHA256}(K_{V_i, \text{RSU}}, m') = \text{MAC}_i^{\text{received}} \quad (3.16)$$

$$|\tau_r - \tau_s| \leq T_b + \epsilon \quad (3.17)$$

$$\text{nonce}_i \notin \mathcal{N}_{\text{seen}} \quad (3.18)$$

where $K_{V_i, \text{RSU}}$ is the session key established via the Kyber+Saber Hybrid-KEM [?], τ_r is the receiver's local clock at reception, and $\mathcal{N}_{\text{seen}}$ is the set of previously observed nonces maintained per sender.

=====

RSU-Managed Logical Key Hierarchy. The session keys $K_{V_i, \text{RSU}}$ used in eq:hmac_computation and the individual signing keys SK_{V_i} used in eq:loc_binding_sig are managed using a Merkle tree structure. Each leaf node ℓ_i stores the individual session key $K_{V_i, \text{RSU}}$ of vehicle V_i , and each internal node j stores a Key Encryption Key (KEK) K_j^{kek} used to encrypt keys of its children. The root stores the group key $K_{\mathcal{G}}$ shared by the RSU and all registered vehicles. When the smart contract in Algorithm 4 executes

REVOKE-SESSION-KEY(V_k), only the d KEKs on the path from leaf ℓ_k to the root require updating, giving a revocation cost of:

$$C_{\text{revoke}} = O(\log n) \quad (3.19)$$

rather than the $O(n)$ cost of naive full re-keying. The updated KEKs are encrypted under the individual keys of the remaining $n-1$ legitimate vehicles and distributed by the RSU within the next beacon interval T_b , ensuring continuity of the HMAC session and threshold signing infrastructure for non-revoked vehicles without interrupting beacon processing. This is directly grounded in RSU-delegated group key management for vehicular multicast [?], where delegating key update responsibility to the RSU significantly reduces rekeying overhead under high vehicle mobility.

...

At each beacon interval t , the SDVN topology is represented as the graph snapshot:

$$\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, \mathbf{X}_t, \mathbf{A}_t) \quad (3.20)$$

where $\mathcal{V}_t$ is the set of active vehicles and RSUs, $\mathcal{E}_t$ is the set of reported topology links, $\mathbf{A}_t \in R^{|\mathcal{V}_t| \times |\mathcal{V}_t|}$ is the weighted adjacency matrix whose entry $\mathbf{A}_{t,uv}$ encodes normalized timestamp freshness of the link report, and $\mathbf{X}_t \in R^{|\mathcal{V}_t| \times d}$ is the node feature matrix. Each row of $\mathbf{X}_t$ is the feature vector of node v :

$$\mathbf{x}_v = [\text{id}_v, \tau_s^{(v)}, c_v^W, \Delta s_v, \rho_v] \in R^d \quad (3.21)$$

where id_v is the node identifier embedding, $\tau_s^{(v)}$ is the last beacon timestamp, c_v^W is the beacon count in sliding window W , Δs_v is the sequence number gap, and ρ_v is the reporter count for adjacent links.

The temporal memory of node v is updated at each event using a Gated Recurrent Unit:

$$\mathbf{m}_v(t) = \text{GRU}(\mathbf{h}_v(t^-), \mathbf{x}_v(t), \phi(\Delta t_v)) \quad (3.22)$$

where $\mathbf{h}_v(t^-)$ is the node embedding at the previous event, $\mathbf{x}_v(t)$ is the current feature vector from eq:node_features, and $\phi(\Delta t_v) = \log(1 + \Delta t_v/T_b)$ is a time-elapsd encoding scaled by the beacon interval T_b to account for SDVN mobility dynamics.

Node embeddings are refined through L rounds of neighbourhood aggregation:

$$\mathbf{h}_v^{(l+1)} = \sigma(\mathbf{W}^{(l)} \cdot \text{MEAN}(\{\mathbf{h}_u^{(l)} \odot \mathbf{A}_{uv}(t) : u \in \mathcal{N}(v, t)\}) + \mathbf{b}^{(l)}) \quad (3.23)$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are trainable weight matrix and bias at layer l , $\sigma(\cdot)$ is the ReLU activation function, $\odot$ denotes element-wise multiplication, and $\mathcal{N}(v, t)$ is the set of neighbours of v in $\mathcal{G}_t$.

The anomaly score for node v at time t is computed from the final-layer embedding:

$$\hat{y}_v(t) = \sigma(\mathbf{w}^\top \mathbf{h}_v^{(L)}(t)) \quad (3.24)$$

where $\mathbf{w} \in R^d$ is a trainable scoring vector and $\sigma(\cdot)$ here denotes the sigmoid function. A full-stack detection alert for node v is raised when $\hat{y}_v(t) > \theta_{\text{FS}}$, where

θ_{FS} is optimized by maximizing the Matthews Correlation Coefficient (MCC) on held-out validation data.

An RSU collects signed beacon reports from n vehicles and constructs a threshold aggregate signature valid only if at least t individual signatures are verified:

$$\text{Verify}(\sigma_{\text{agg}}, \text{PK}_{\text{agg}}) = 1 \iff |\{i : \text{Verify}(\sigma_i, \text{msg}_i, \text{PK}_{V_i}) = 1\}| \geq t \quad (3.25)$$

where $t \geq \lfloor n/2 \rfloor + 1$ is the threshold set to prevent single-node compromise and σ_{agg} is constructed using the lattice-based aggregate scheme of [?].

To prevent false witness attestation in the ME attack, each topology observation report is constructed by binding the observed link to the reporter's GPS-attested position and RSSI measurement before signing:

$$m'_{V_k} = e_{ij} \parallel \text{pos}_{V_k} \parallel \text{RSSI}_{V_k \leftarrow V_i} \parallel \tau_s \parallel \text{nonce} \quad (3.26)$$

$$\sigma_{V_k} = \text{Sign}(SK_{V_k}, m'_{V_k}) \quad (3.27)$$

where SK_{V_k} is the private signing key of V_k established via the Kyber+Saber Hybrid-KEM [?], pos_{V_k} is the GPS-attested position of V_k at time τ_s , and $\text{RSSI}_{V_k \leftarrow V_i}$ is the received signal strength from V_i measured at V_k .

The controller accepts a single topology observation report from V_k for link e_{ij} if and only if the signature is valid and the physical consistency conditions are jointly satisfied:

$$\text{Accept}_{V_k}(e_{ij}) \iff \{ \text{Verify}(\sigma_{V_k}, \text{PK}_{V_k}) = 1 \mid \text{pos}_{V_k}, e_{ij} \leq r_{\text{comm}} \text{RSSI}_{V_k \leftarrow V_i} \geq \text{RSSI}_{\min}(r_{\text{comm}}) \} \quad (3.28)$$

To resist collusion among distant attackers, link e_{ij} is accepted into the controller topology only when a quorum of at least t physically-consistent witnesses independently confirm it:

$$\text{Accept}(e_{ij}) \iff \left| \{V_k : \text{Accept}_{V_k}(e_{ij}) = 1\} \right| \geq t \quad (3.29)$$

where $\text{Accept}_{V_k}(e_{ij})$ is defined in eq:me_acceptsingleandisthesamethresholdasin eq : threshold_s_i, consistencychecking.

3.2 System Architecture

This research proposes a comprehensive, hierarchical security framework designed to secure Software-Defined Vehicular Networks (SDVNs) against sophisticated temporal-echo attacks. The architecture necessitates a defense mechanism that transcends traditional rule-based filtering by enforcing temporal consistency and immutable accountability within the control plane. As illustrated in Figure 3.1, the system is structured into six logical layers that bridge the gap between experimental simulation and cross-domain fraud analytics

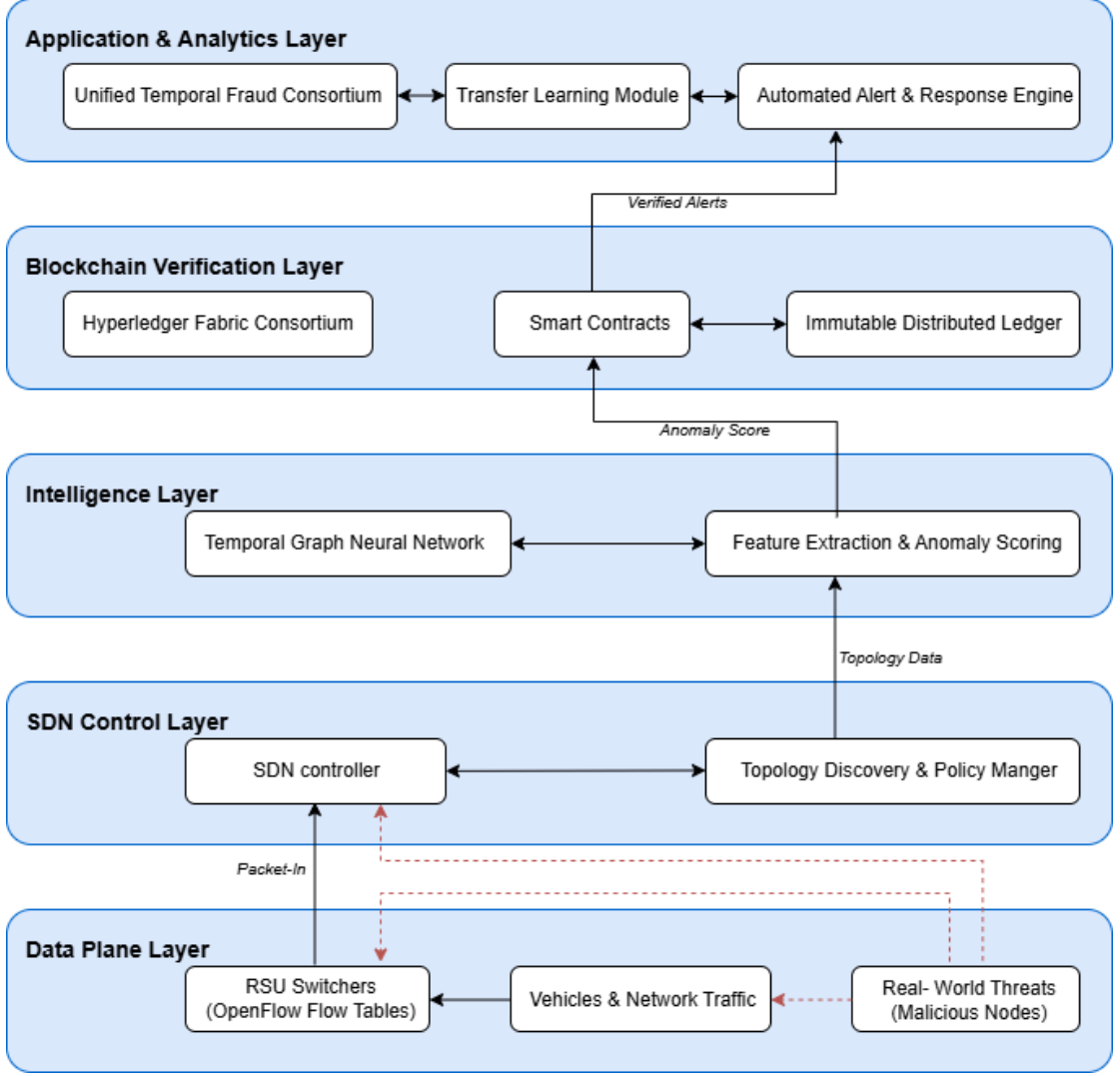


Figure 3.1: Proposed Layered System Architecture for Secure Temporal Anomaly Detection in SDVN

3.2.1 Data Plane Layer

The **Data Plane Layer** serves as the foundational tier of the proposed framework, representing the physical forwarding infrastructure of the SDVN. This layer comprises vehicles equipped with On-Board Units (OBUs) and Roadside Units (RSUs) that function as OpenFlow-enabled switches. These devices are responsible for forwarding network traffic based exclusively on flow tables provided by the control plane, acting as the physical entry point for both legitimate traffic and malicious topology discovery packets injected by compromised nodes.

While the proposed architecture is designed for real-world deployment, for the purpose of this research, this layer is experimentally modeled using **NS-3** and **SUMO**. These tools generate realistic vehicular mobility patterns and network traffic, allowing for the systematic injection of specific temporal threats—Topology

Time-Warp (TTW), Beacon State Heartbeat Hijack (BSHH), and Multipath Echo (ME)—to train and validate the detection models.

3.2.2 Control and Intelligence Layers

The SDN Control Layer houses a centralized controller (Ryu or ONOS) that tracks the global network state. Rather than directly processing topology updates, it collects raw Link Layer Discovery Protocol (LLDP) events and forwards them through a Northbound API to the Intelligence Layer for scrutiny.

Detection logic lives in the Intelligence Layer. A **Temporal Graph Neural Network (TGN)** or **GraphSAGE model** extracts features and scores anomalies. It transforms network events into temporal graph snapshots, flagging timestamp disorders and link persistence issues typical of temporal-echo attacks.

3.2.3 Verification and Application Layers

The Blockchain Verification Layer uses Hyperledger Fabric for audit trails and fault tolerance. Smart Contracts verify Intelligence Layer alerts, log attack metadata to an immutable ledger, and trigger automatic isolation of malicious nodes.

The top Application Analytics Layer shows framework extensibility. Its Transfer Learning Module applies vehicular temporal features to detect financial fraud (tax evasion) in unrelated domains, proving shared structures in deception patterns across fields.

3.3 High-Level Functional Workflow

While the system architecture outlines structural layers, Figure 3.2 depicts the operational data pipeline as a functional block diagram. This workflow converts live network traffic into security actions across four processing stages.

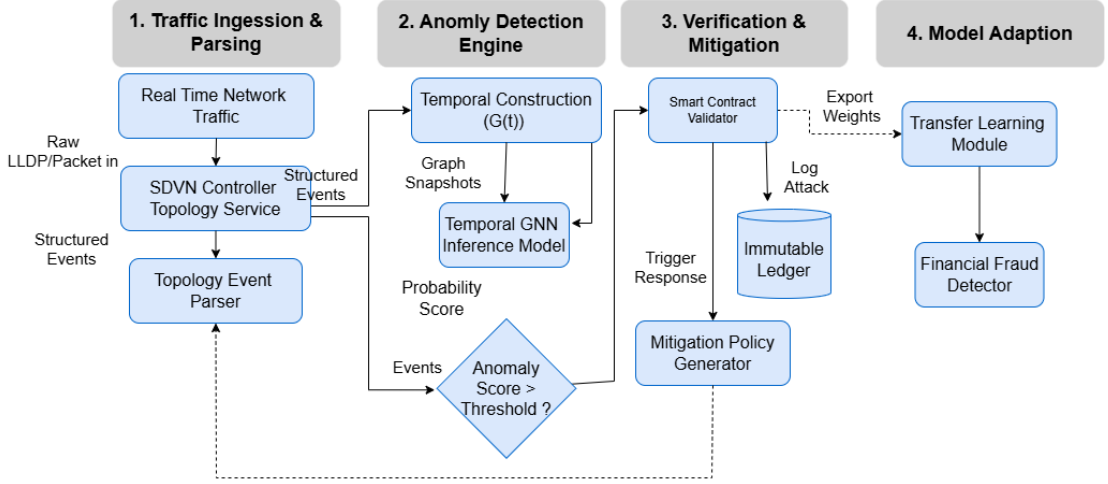


Figure 3.2: Proposed Attack Detection and Mitigation Framework Workflow

As shown in figure 3.2, traffic Ingestion Parsing kicks off the process by capturing real-time flows. The SDN Controller collects control-plane messages and passes them to a Topology Event Parser. This module strips noise, pulls key events (Packet-In, LLDP), and shapes raw packets into structured topology records.

Next, the Anomaly Detection Engine takes over. A Temporal Graph Construction unit assembles dynamic graph sequences (G_t) tracking network evolution. The Temporal GNN Inference Model then scores each snapshot’s anomaly probability against a trained threshold—flagging states that exceed it.

Detection triggers Verification Mitigation. A Smart Contract Validator checks alert validity, then splits execution: attack details hit the Immutable Ledger for audits, while a Mitigation Policy Generator crafts isolation rules (port blocks) pushed back through the SDN Controller.

Model Adaptation caps the pipeline for cross-domain use. GNN weights export periodically to a Transfer Learning Module, seeding a Financial Fraud Detector that spots tax evasion patterns mirroring the network’s temporal attack structures.

3.4 Formulation of Temporal-Echo Attacks in SD-VNs

3.4.1 Topology Time Warp (TTW) Attack

Unlike link fabrication attacks, the topology time warp attack preserves spatial correctness while corrupting temporal ordering, making it particularly difficult to detect using conventional topology validation mechanisms. The TTW attack targets the temporal consistency of topology information in SDVNs rather than introducing false links.

TTW Attack via Malicious Vehicle

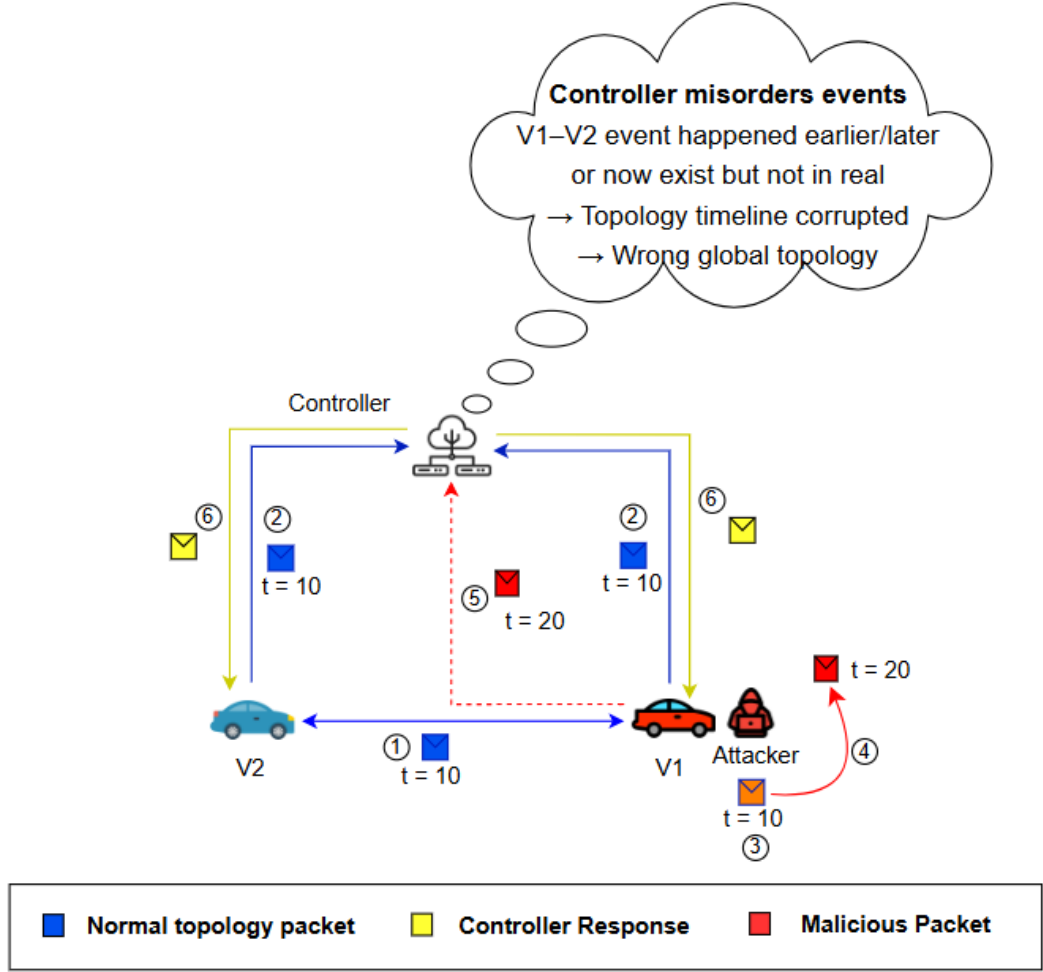


Figure 3.3: TTW Attack via Malicious Vehicle (Without RSU): V1 manipulates timestamps of HELLO messages sent to the controller.

Topology Time Warp Attack (Malicious Vehicle) The Topology Time-Warp (TTW) Attack via malicious vehicle (without RSU) occurs when a compromised vehicle delays and replays legitimate topology updates with forged timestamps, causing the controller to perceive stale links as currently active (Fig. 3.3). This temporal manipulation creates inconsistencies between the real network state and the controller's view without altering physical connectivity.

The attack proceeds as follows:

1. **V2V Neighbor Discovery:** At time $t = 10$, V2 broadcasts a HELLO beacon that is received by V1, allowing V1 to detect V2 as a neighboring vehicle. V2 $\rightarrow$ V1: HELLO V1 $\rightarrow$ V2: HELLO

2. **Topology Update:** Based on the V2V HELLO exchange, both V1 and V2 independently generate legitimate topology updates. V1 sends $\langle V1 \text{ sees } V2, t = 10 \rangle$ and V2 sends $\langle V2 \text{ sees } V1, t = 10 \rangle$ to the controller, which records the bidirectional V1–V2 link as active. V1 $\rightarrow$ Controller: $\langle V1 \text{ sees } V2, t = 10 \rangle$ V2 $\rightarrow$ Controller: $\langle V2 \text{ sees } V1, t = 10 \rangle$
3. **Malicious Vehicle Stores Legitimate Update:** The malicious vehicle intercepts and stores a copy of the legitimate topology update for later replay. Old packet: $\langle V1 \text{ sees } V2, t = 10 \rangle$
4. **Link Failure and Attack Execution:** At time $t = 15$, vehicles V1 and V2 move out of communication range and the physical link breaks. At $t = 20$, the malicious vehicle replays the old legitimate topology update with a forged current timestamp, causing the controller to incorrectly believe the V1–V2 link is still active. Forged packet: $\langle V1 \text{ sees } V2, t = 20 \rangle$
5. **Faulty Routing Decisions:** The controller installs routing rules based on the forged "fresh" topology update, directing traffic through the non-existent V1–V2 link, resulting in packet loss and routing instability.

Topology Time-Warp Attack (Malicious Vehicle, No RSU): An attack where a compromised vehicle replays legitimate topology updates with forged timestamps after physical link failure, creating false perceptions of network connectivity and poisoning controller routing decisions.

TTW Attack via Malicious RSU

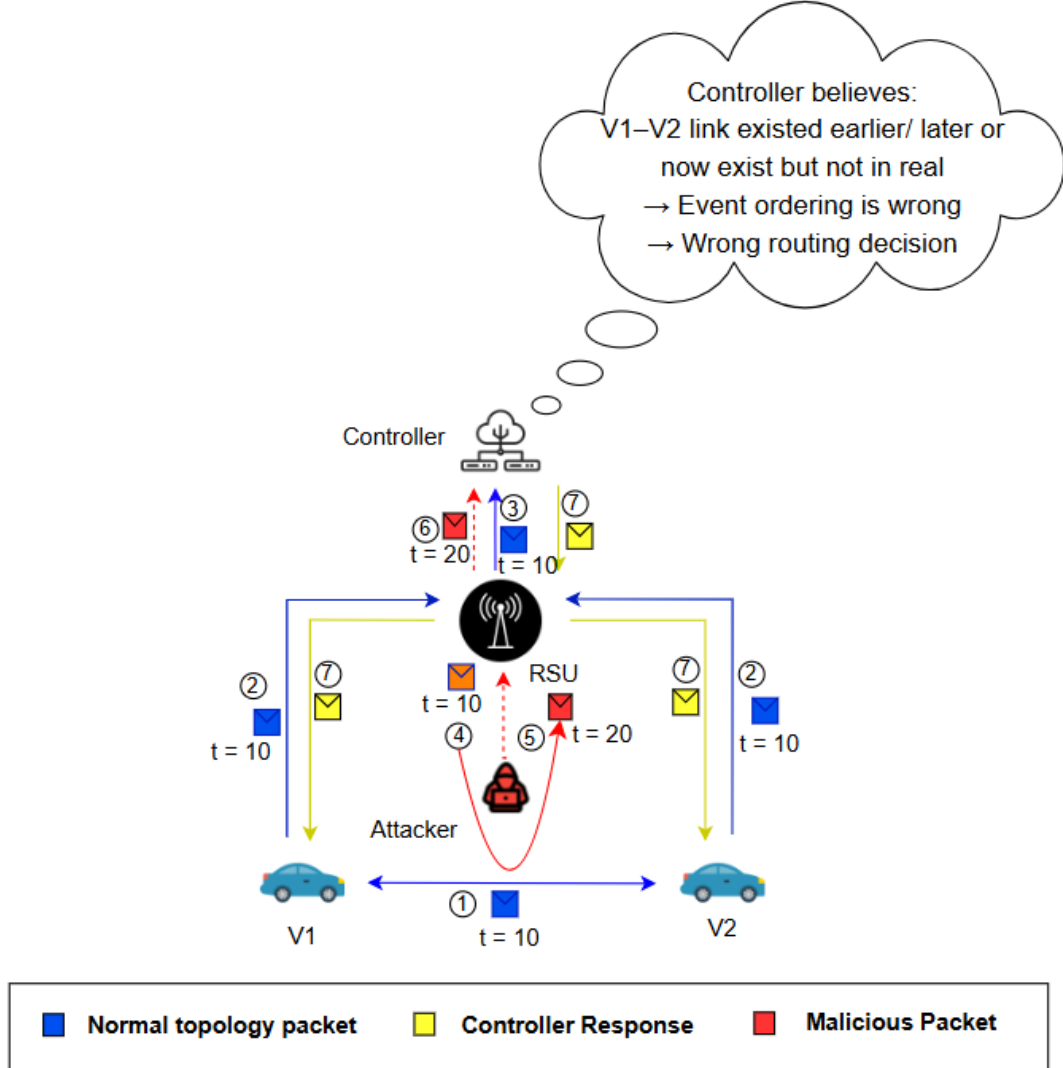


Figure 3.4: TTW Attack via Malicious RSU: RSU manipulates timestamps in forwarded vehicle HELLO messages.

Topology Time Warp Attack (Malicious RSU) The Topology Time-Warp (TTW) Attack via malicious RSU occurs when a compromised RSU intercepts legitimate topology updates, stores them, and later replays them with forged timestamps after physical link failure, causing the controller to perceive stale links as currently active (Fig. 3.4). This temporal manipulation creates inconsistencies between the real network state and the controller’s view without altering physical connectivity.

The attack proceeds as follows:

1. **V2V Neighbor Discovery:** At time $t = 10$, V2 broadcasts a HELLO

beacon that is received by V1, allowing V1 to detect V2 as a neighboring vehicle. V2 → V1: HELLO V1 → V2: HELLO

2. **Topology Update to RSU:** Based on the V2V HELLO exchange, both V1 and V2 independently generate legitimate topology updates and send them to the RSU. V1 → RSU: $\langle V1 \text{ sees } V2, t = 10 \rangle$ V2 → RSU: $\langle V2 \text{ sees } V1, t = 10 \rangle$
3. **RSU Aggregation and Forwarding:** The RSU aggregates the received topology updates $\langle V1 \text{ sees } V2, t = 10 \rangle$ and $\langle V2 \text{ sees } V1, t = 10 \rangle$ and forwards them to the controller, establishing the bidirectional V1–V2 link as active.
4. **Malicious RSU Stores Legitimate Update:** The malicious RSU stores a copy of the legitimate topology update for later replay. Old packet: $\langle V1 \text{ sees } V2, t = 10 \rangle$
5. **Link Failure and Attack Preparation:** At time $t = 15$, vehicles V1 and V2 move out of communication range and the physical link breaks. At $t = 20$, the malicious RSU forges the stored legitimate topology update by modifying its timestamp from $\langle V1 \text{ sees } V2, t = 10 \rangle$ to $\langle V1 \text{ sees } V2, t = 20 \rangle$.
6. **Replay of Forged Update:** The malicious RSU replays the timestamp-forged topology update to the controller, causing stale connectivity information to appear fresh. RSU → Controller: $\langle V1 \text{ sees } V2, t = 20 \rangle$
7. **Faulty Routing Decisions:** The controller installs routing rules based on the forged "fresh" topology update, directing traffic through the non-existent V1–V2 link, resulting in packet loss and routing instability.

Topology Time-Warp Attack (Malicious RSU): An attack where a compromised RSU replays stored legitimate topology updates with forged timestamps after physical link failure, creating false perceptions of active connectivity and poisoning controller routing decisions.

TTW Attack via Malicious Controller

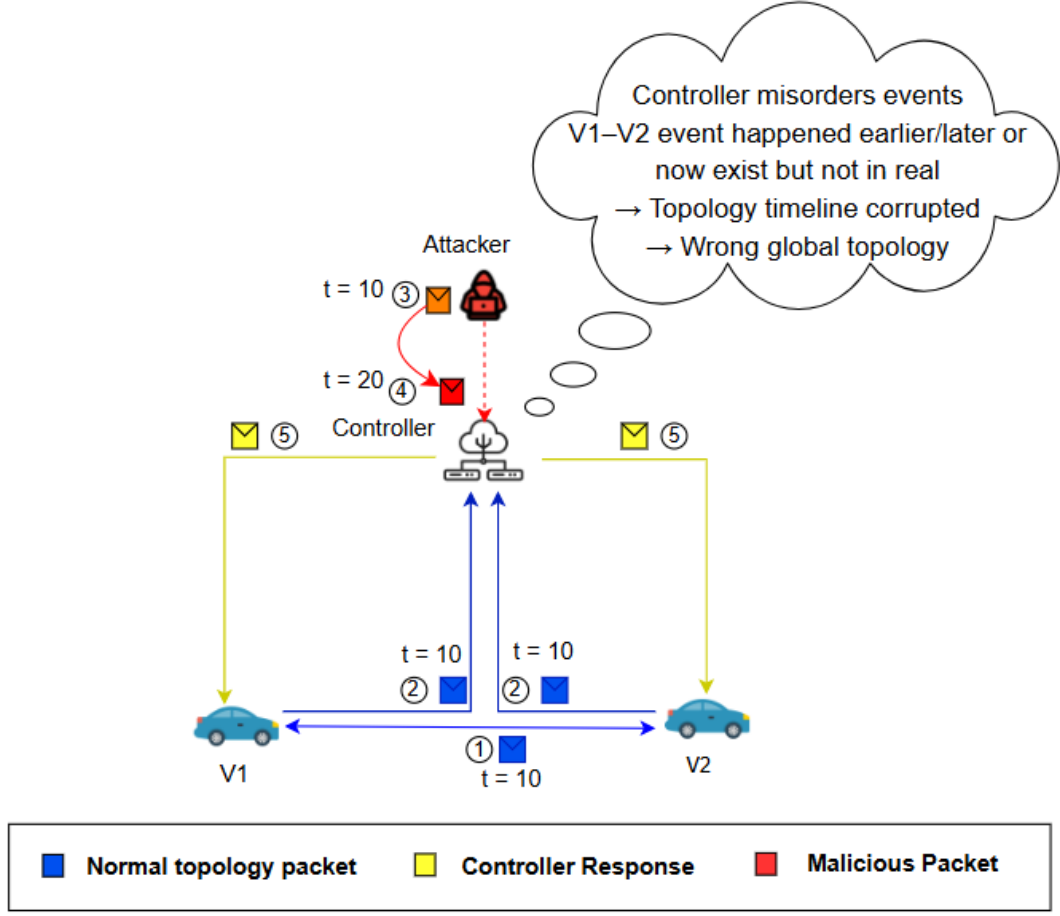


Figure 3.5: TTW Attack via Malicious Controller (Without RSU): The controller internally manipulates timestamps of received HELLO messages.

Topology Time Warp Attack (Malicious Controller – Timestamp Injection, No RSU) The Topology Time-Warp (TTW) Attack via malicious controller occurs when a compromised controller stores legitimate topology updates and later replays them with forged timestamps after physical link failure, causing itself to perceive stale links as currently active (Fig. 3.5). This internal temporal manipulation creates inconsistencies between the real network state and the controller’s topology database without altering physical connectivity.

The attack proceeds as follows:

1. **V2V Neighbor Discovery:** At time $t = 10$, V2 broadcasts a HELLO beacon that is received by V1, allowing V1 to detect V2 as a neighboring vehicle. V2 → V1: HELLO V1 → V2: HELLO
2. **Topology Update:** Based on the V2V HELLO exchange, both V1 and V2 independently generate legitimate topology updates and send them directly

to the controller, which records the bidirectional V1–V2 link as active. V1 → Controller: < V1 sees V2, $t = 10$ > V2 → Controller: < V2 sees V1, $t = 10$ >

3. **Malicious Controller Stores Legitimate Update:** The malicious controller stores a copy of the received legitimate topology update for later replay. Old packet: < V1 sees V2, $t = 10$ >
4. **Link Failure and Attack Preparation:** At time $t = 15$, vehicles V1 and V2 move out of communication range and the physical link breaks. At $t = 20$, the malicious controller forges the stored legitimate topology update by modifying its timestamp to appear current. Forged packet: < V1 sees V2, $t = 20$ >
5. **Replay into Topology Database:** The controller reinserts the forged topology update into its internal topology database as a "fresh" observation, causing itself to incorrectly believe the V1–V2 link remains active.
6. **Faulty Routing Decisions:** The controller installs routing rules based on the forged "fresh" topology update, directing traffic through the non-existent V1–V2 link, resulting in packet loss and routing instability.

Topology Time-Warp Attack (Malicious Controller): An attack where a compromised controller internally replays stored legitimate topology updates with forged timestamps after physical link failure, creating false perceptions of active connectivity and poisoning its own routing decisions.

TTW Attack via Malicious Controller

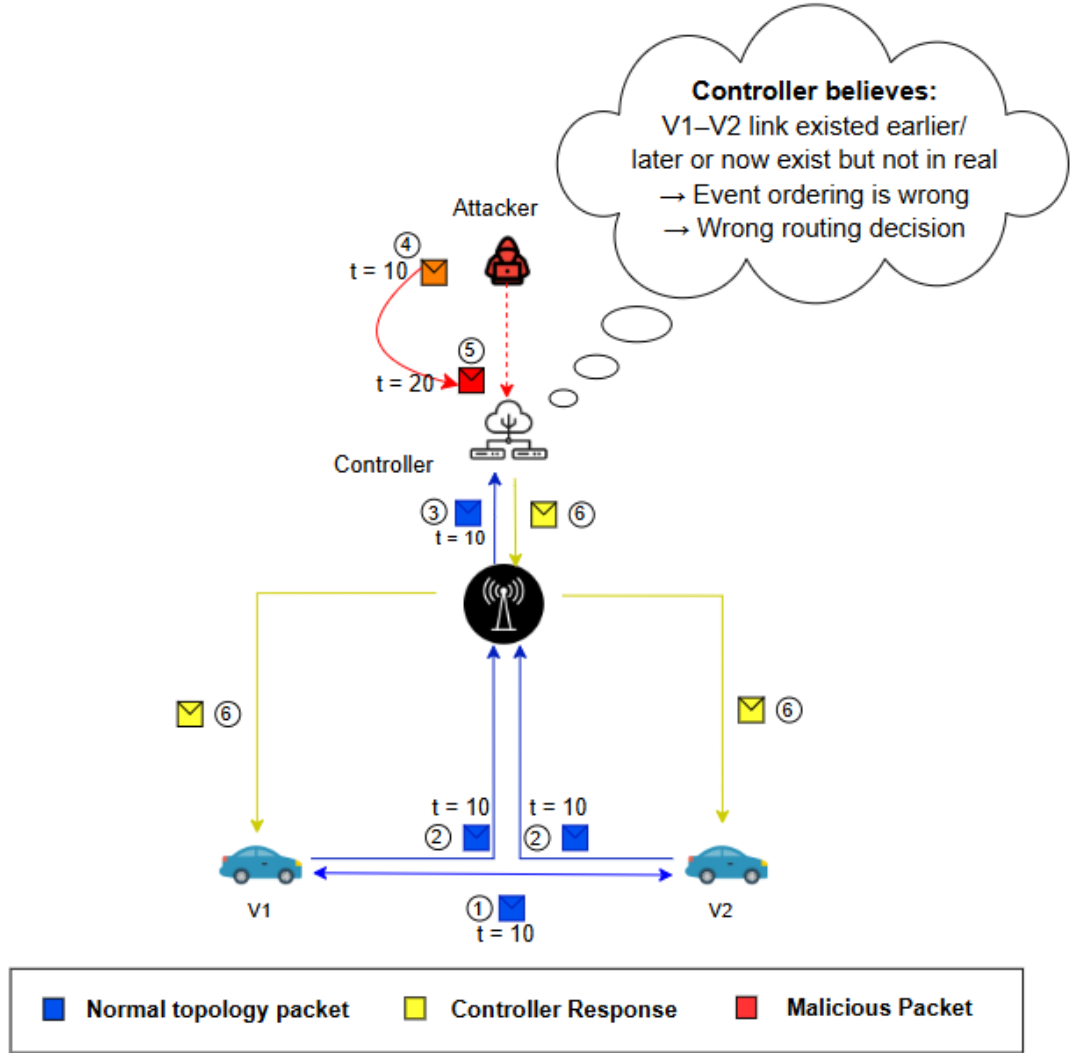


Figure 3.6: TTW Attack via Malicious Controller (With RSU): The controller internally manipulates timestamps of received HELLO messages.

Topology Time Warp Attack (Malicious Controller – Reorder/Replay), with RSU The Topology Time-Warp (TTW) Attack via malicious controller (with RSU) occurs when a compromised controller stores legitimate topology updates received through RSU aggregation and later replays them with forged timestamps after physical link failure, causing itself to perceive stale links as currently active (Fig. 3.6). This internal temporal manipulation creates inconsistencies between the real network state and the controller’s topology database without altering physical connectivity.

The attack proceeds as follows:

1. **V2V Neighbor Discovery:** At time $t = 10$, V2 broadcasts a HELLO

beacon that is received by V1, allowing V1 to detect V2 as a neighboring vehicle. V2 → V1: HELLO V1 → V2: HELLO

2. **Topology Update to RSU:** Based on the V2V HELLO exchange, both V1 and V2 independently generate legitimate topology updates and send them to the RSU. V1 → RSU: $\langle V1 \text{ sees } V2, t = 10 \rangle$ V2 → RSU: $\langle V2 \text{ sees } V1, t = 10 \rangle$
3. **RSU Aggregation and Forwarding:** The RSU aggregates the received topology updates $\langle V1 \text{ sees } V2, t = 10 \rangle$ and $\langle V2 \text{ sees } V1, t = 10 \rangle$ and forwards them to the controller, establishing the bidirectional V1–V2 link as active.
4. **Malicious Controller Stores Legitimate Update:** The malicious controller stores a copy of the received legitimate topology update (via RSU) for later replay. Old packet: $\langle V1 \text{ sees } V2, t = 10 \rangle$
5. **Link Failure and Attack Preparation:** At time $t = 15$, vehicles V1 and V2 move out of communication range and the physical link breaks. At $t = 20$, the malicious controller forges the stored legitimate topology update by modifying its timestamp to appear current. Forged packet: $\langle V1 \text{ sees } V2, t = 20 \rangle$
6. **Replay into Topology Database:** The controller reinserts the forged topology update into its internal topology database as a "fresh" observation, causing itself to incorrectly believe the V1–V2 link remains active.
7. **Faulty Routing Decisions:** The controller installs routing rules based on the forged "fresh" topology update, directing traffic through the non-existent V1–V2 link, resulting in packet loss and routing instability.

Topology Time-Warp Attack (Malicious Controller with RSU): An attack where a compromised controller replays stored legitimate topology updates (received via RSU aggregation) with forged timestamps after physical link failure, creating false perceptions of active connectivity and poisoning its own routing decisions.

3.4.2 Beacon State Heartbeat Hijack Attack (BSHH) Attack

The Beacon State Heartbeat Hijack (BSHH) attack undermines SDVN topology inference by tampering with heartbeat identity fields, while leaving payloads untouched and avoiding new message injections. It capitalizes on the inherent trust in heartbeat signals, which vehicles rely on to discover neighbors and sustain consistent global topology views.

BSHH Attack via Malicious Vehicle (Without RSU)

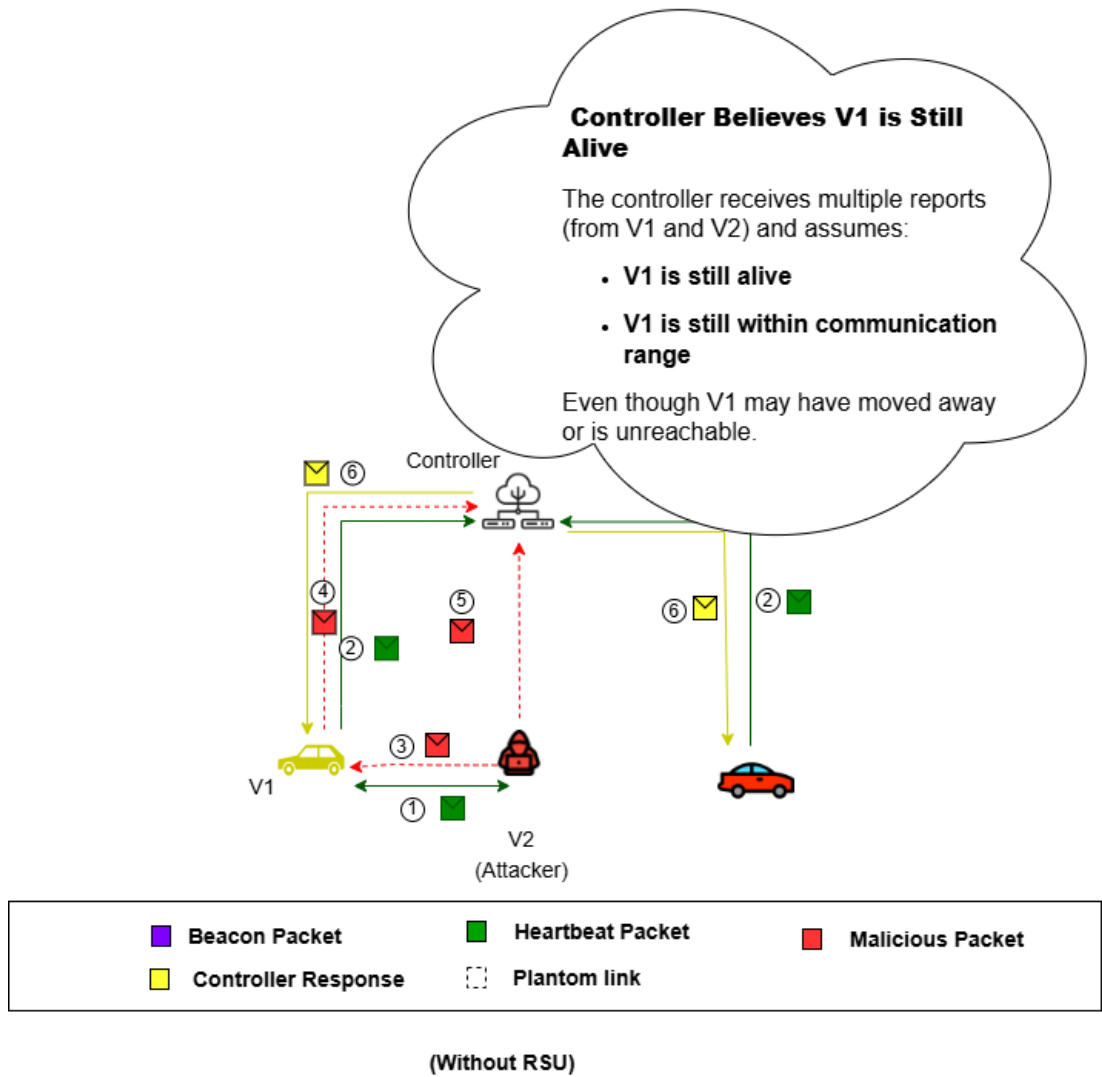


Figure 3.7: BSHH Attack via Malicious Vehicle (Without RSU): V2 hijacks heartbeat messages from V3 to V1, inducing false adjacency.

Beacon State Heartbeat Hijack Attack (Malicious Vehicle) The Beacon State Heartbeat Hijack (BSHH) Attack via malicious vehicle (without RSU) occurs when a compromised vehicle (V2) intercepts legitimate heartbeat exchanges, stores old heartbeats, and replays them to both peer vehicles and the controller—creating false liveness perceptions and hijacking the network state (Fig. 3.7). This zero-payload temporal attack maintains stale topology perceptions without fabricating new links.

The attack proceeds as follows:

1. **Normal Heartbeat Exchange ($t = 5$):** Vehicles V1 and V2 exchange heartbeat beacons normally. V1 $\rightarrow$ V2: "I am alive" ($t = 5$) V2 $\rightarrow$ V1: "I am alive" ($t = 5$)
2. **Both Forward Legitimate Heartbeats to Controller ($t = 5$):** Both vehicles send honest heartbeat reports to the controller. V1 $\rightarrow$ Controller: Heartbeat (Sender = V1, Timestamp = 5) V2 $\rightarrow$ Controller: Heartbeat (Sender = V2, Timestamp = 5)
3. **V2 Replays Old Heartbeat to V1 ($t = 10$):** V2 (malicious) stores an old heartbeat from V1 (from $t = 0$) and replays it to V1 at $t = 10$. V2 $\rightarrow$ V1: Heartbeat (Sender = V1, Timestamp = 0) (old legitimate packet replayed)
4. **V1 Forwards Old Heartbeat to Controller:** V1 believes the received heartbeat is valid and forwards it to the controller. V1 $\rightarrow$ Controller: Heartbeat (Sender = V1, Timestamp = 0)
5. **V2 Also Sends Old Heartbeat to Controller (Hijack):** V2 sends the same old heartbeat directly to the controller, impersonating V1. V2 $\rightarrow$ Controller: Heartbeat (Sender = V1, Timestamp = 0) (fake packet — impersonating V1)
6. **Faulty Routing Decisions:** Using the conflicting heartbeat timestamps and incorrect liveness information, the controller installs wrong routes: Packets routed through non-existent/stale links; Packet loss and delays occur; Network performance degrades significantly.

Beacon State Heartbeat Hijack (Malicious Vehicle, No RSU): An attack where a compromised vehicle replays stored legitimate heartbeats to both peers and controller—impersonating liveness through zero-payload temporal hijacking, poisoning network state without fabricating structural elements.

BSHH Attack via Malicious RSU

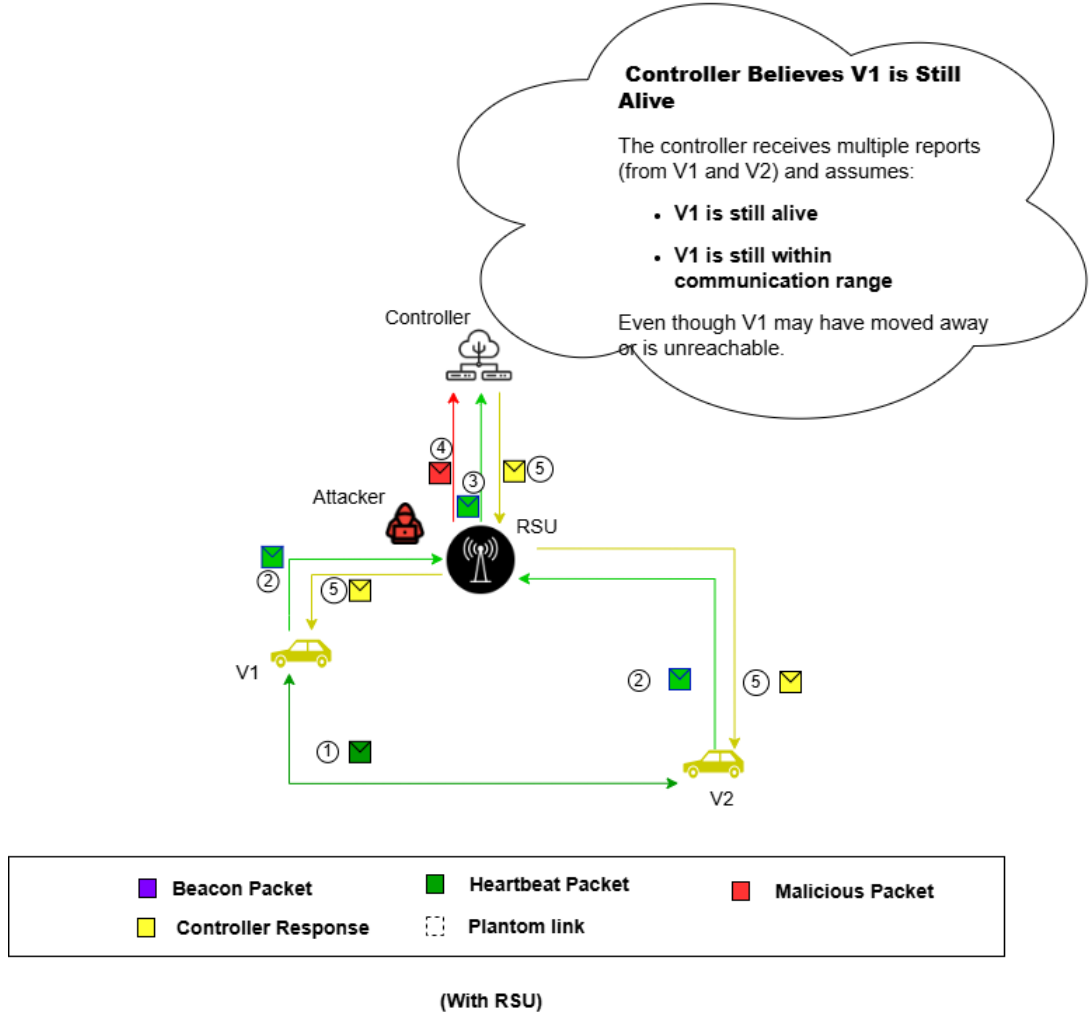


Figure 3.8: BSHH Attack via Malicious RSU: The RSU manipulates the sender identity of heartbeat messages to induce false adjacency.

Beacon State Heartbeat Hijack Attack (Malicious RSU) The Beacon State Heartbeat Hijack (BSHH) Attack via malicious RSU occurs when a compromised RSU intercepts legitimate heartbeat exchanges from vehicles, stores old heartbeats, and replays them to the controller—creating false liveness perceptions and hijacking the network state through aggregated topology poisoning (Fig. 3.8). This zero-payload temporal attack maintains stale topology perceptions without fabricating new links.

The attack proceeds as follows:

1. **Normal Heartbeat Exchange ($t = 5$):** Vehicles V1 and V2 exchange heartbeat beacons normally. V1 $\rightarrow$ V2: "I am alive" ($t = 5$) V2 $\rightarrow$ V1: "I am alive" ($t = 5$)

2. **Vehicles Forward Legitimate Heartbeats to RSU ($t = 5$):** Both vehicles send honest heartbeat reports to the RSU. $V1 \rightarrow \text{RSU}$: Heartbeat (Sender = V1, Timestamp = 5) $V2 \rightarrow \text{RSU}$: Heartbeat (Sender = V2, Timestamp = 5)
3. **RSU Forwards Aggregated Heartbeat to Controller ($t = 5$):** The RSU aggregates and forwards legitimate heartbeat summary to the controller. $\text{RSU} \rightarrow \text{Controller}$: [V1($t=5$), V2($t=5$)]
4. **RSU Replays Old Heartbeat ($t = 10$):** The malicious RSU stores and replays an old legitimate heartbeat (from $t = 0$) as if current, impersonating V1. $\text{RSU} \rightarrow \text{Controller}$: Heartbeat (Sender = V1, Timestamp = 0) (old legitimate packet replayed, sent via RSU using V1's identity)
5. **Conflicting Heartbeat Hijack:** The RSU sends additional conflicting old heartbeats, creating timestamp inconsistencies and false liveness signals. $\text{RSU} \rightarrow \text{Controller}$: Heartbeat (Sender = V1, Timestamp = 0) (duplicate attack packet via impersonation)
6. **Faulty Routing Decisions:** Using the conflicting heartbeat timestamps and incorrect liveness information from RSU aggregation, the controller installs wrong routes: Packets routed through non-existent/stale links; Packet loss and delays occur; Network performance degrades significantly.

Beacon State Heartbeat Hijack (Malicious RSU): An attack where a compromised RSU replays stored legitimate heartbeats through aggregated topology reports—impersonating vehicle liveness via zero-payload temporal hijacking, poisoning controller network state perception.

BSHH Attack via Malicious Controller, without RSU

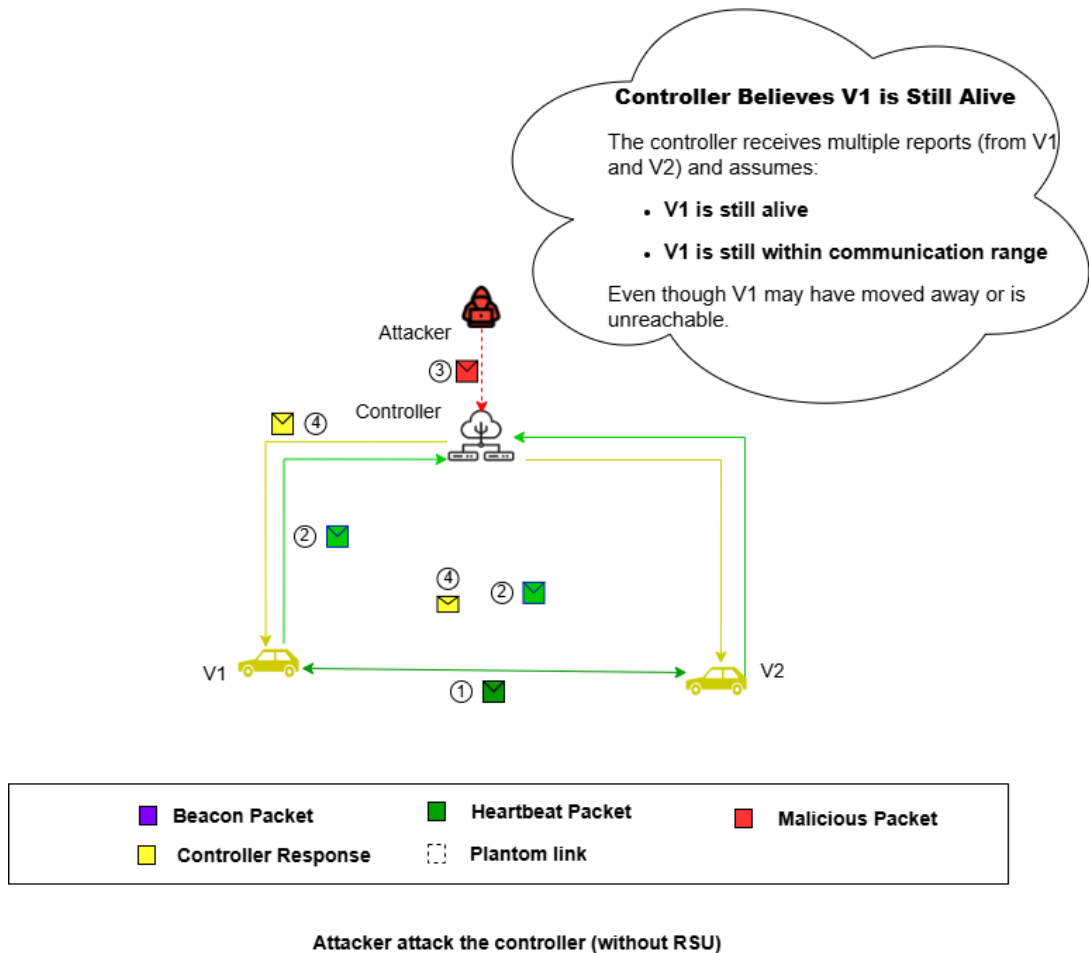


Figure 3.9: BSHH Attack via Malicious Controller (Without RSU): The controller internally hijacks heartbeat identities to induce false adjacency.

The Beacon State Heartbeat Hijack (BSHH) Attack via malicious controller (without RSU) occurs when a compromised controller stores legitimate heartbeat reports from vehicles and internally treats old heartbeats as current—creating false liveness perceptions and hijacking its own network state view (Fig. 3.9). This internal zero-payload temporal attack maintains stale topology perceptions without external communication or fabricating new links.

The attack proceeds as follows:

1. **Normal Heartbeat Exchange ($t = 5$):** Vehicles V1 and V2 exchange heartbeat beacons normally. $V1 \rightarrow V2$: "I am alive" ($t = 5$) $V2 \rightarrow V1$: "I am alive" ($t = 5$)
2. **Both Forward Legitimate Heartbeats to Controller ($t = 5$):** Both vehicles send honest heartbeat reports directly to the controller. $V1 \rightarrow$

Controller: Heartbeat (Sender = V1, Timestamp = 5) V2 → Controller:
Heartbeat (Sender = V2, Timestamp = 5)

3. **Controller Stores Old Heartbeats:** The malicious controller maintains a store of previously received old heartbeats from V1 and V2. Stored packets: Heartbeat (Sender = V1, Timestamp = 0) Heartbeat (Sender = V2, Timestamp = 0)
4. **Controller Maliciously Treats Old Heartbeats as Fresh:** The malicious controller internally reprocesses these old heartbeats as if they were current topology updates, overriding newer legitimate information and creating false liveness signals in its topology database.
5. **Internal State Hijacking:** By treating stale heartbeats as active, the controller maintains incorrect vehicle liveness status—believing V1 and V2 remain connected and operational despite physical mobility changes.
6. **Faulty Routing Decisions:** Using the manipulated heartbeat-based topology, the controller installs wrong routes: Packets routed through non-existent/stale links; Packet loss and delays occur; Network performance degrades significantly.

Beacon State Heartbeat Hijack (Malicious Controller, No RSU): An attack where a compromised controller internally reprocesses stored old legitimate heartbeats as current topology updates—creating false liveness through zero-payload temporal state hijacking without external communication.

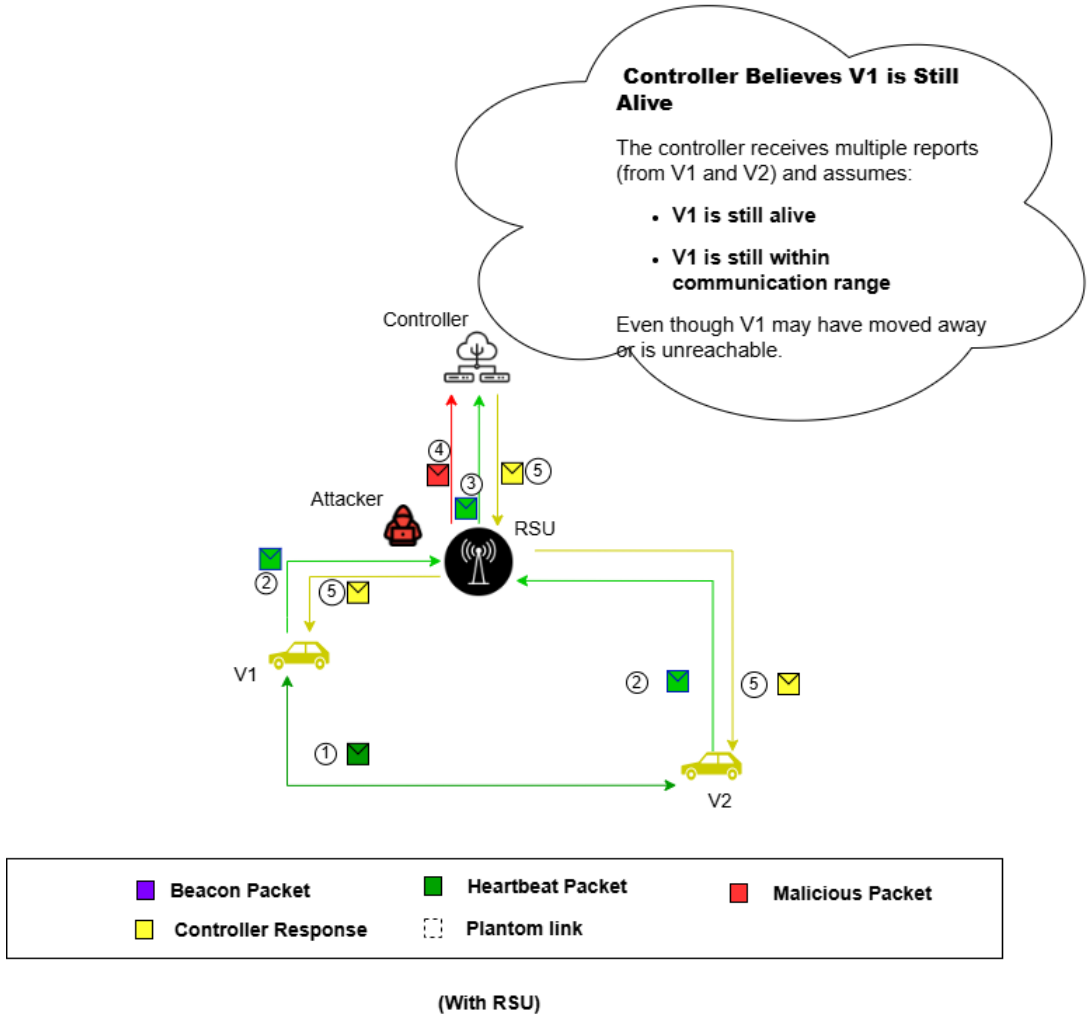


Figure 3.10: BSHH Attack via Malicious Controller (With RSU): The controller internally hijacks heartbeat identities to induce false adjacency.

Beacon State Heartbeat Hijack Attack (Malicious Controller – Direct Injection) The Beacon State Heartbeat Hijack (BSHH) Attack via malicious controller (direct injection with RSU) occurs when a compromised controller receives legitimate heartbeat reports through RSU aggregation, stores old heartbeats, and internally treats them as current—creating false liveness perceptions and hijacking its network state view (Fig. 3.10). This internal zero-payload temporal attack maintains stale topology perceptions without external communication or fabricating new links.

The attack proceeds as follows:

1. **Normal Heartbeat Exchange ($t = 5$):** Vehicles V1 and V2 exchange heartbeat beacons normally. V1 $\rightarrow$ V2: "I am alive" ($t = 5$) V2 $\rightarrow$ V1: "I am alive" ($t = 5$)
2. **Both Forward Legitimate Heartbeats to RSU ($t = 5$):** Both vehicles send honest heartbeat reports to the RSU. V1 $\rightarrow$ RSU: Heartbeat (Sender

= V1, Timestamp = 5) V2 → RSU: Heartbeat (Sender = V2, Timestamp = 5)

3. **RSU Aggregates and Forwards to Controller (t = 5):** The RSU aggregates legitimate heartbeat reports and forwards the summary to the controller. RSU → Controller: [V1(t=5), V2(t=5)]
4. **Controller Stores Old Heartbeats:** The malicious controller maintains a store of previously received old heartbeats from V1 and V2 (via prior RSU aggregation). Stored packets: Heartbeat (Sender = V1, Timestamp = 0) Heartbeat (Sender = V2, Timestamp = 0)
5. **Controller Maliciously Treats Old Heartbeats as Fresh:** The malicious controller internally reprocesses these old heartbeats (received via RSU) as if they were current topology updates, overriding newer legitimate RSU-forwarded information and creating false liveness signals.
6. **Faulty Routing Decisions:** Using the manipulated heartbeat-based topology from stale RSU-aggregated data, the controller installs wrong routes: Packets routed through non-existent/stale links; Packet loss and delays occur; Network performance degrades significantly.

Beacon State Heartbeat Hijack (Malicious Controller with RSU): An attack where a compromised controller internally reprocesses stored old legitimate heartbeats (received via RSU aggregation) as current topology updates—creating false liveness through zero-payload temporal state hijacking.

3.4.3 Multipath Echo (ME) Attack

The Multipath Echo (ME) attack exploits duplicated topology observations to mislead the SDN controller into inferring multiple logical paths that do not exist in the physical network. By replaying or reinterpreting legitimate topology messages, an attacker can induce false path diversity and disrupt routing and forwarding decisions.

ME Attack via Malicious Vehicles

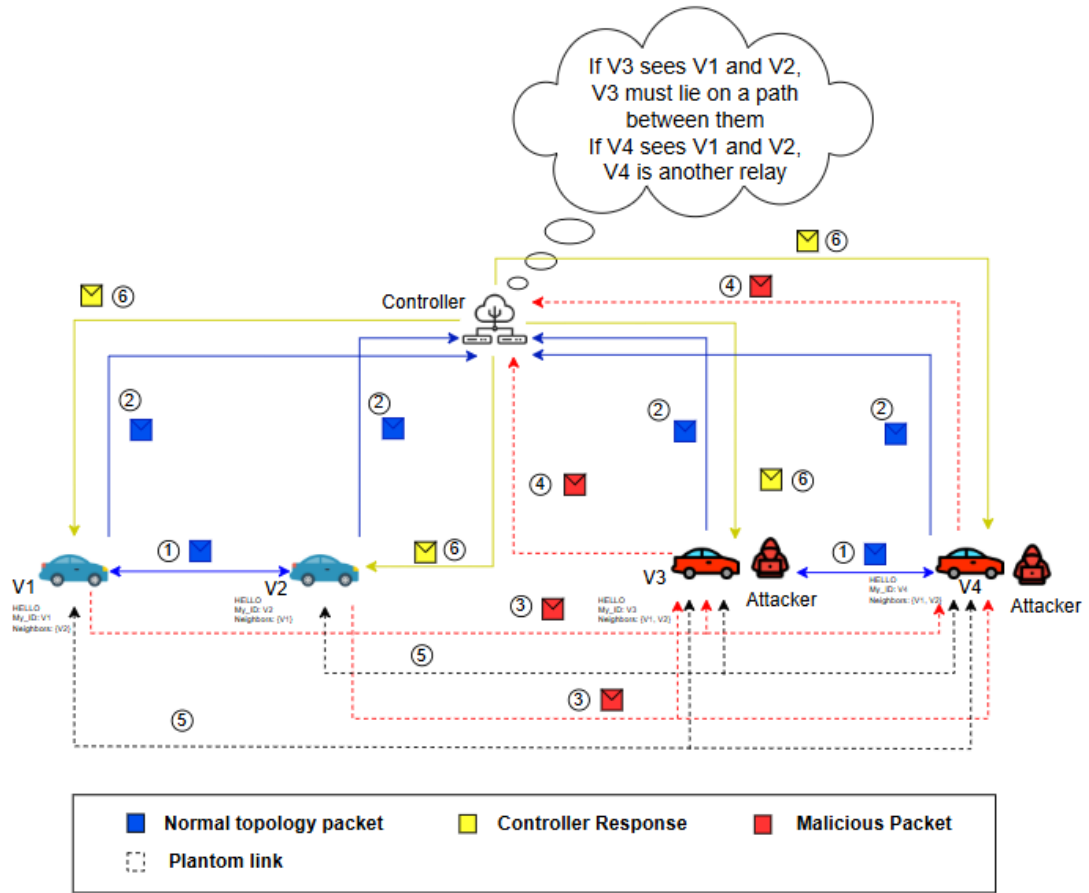


Figure 3.11: Vehicle-based Multipath Echo Attack (Without RSU): V3 and V4 replay V1–V2 topology to the controller, creating false paths.

ME Attack Steps (Vehicle Overhearing Variant) The Multipath Echo (ME) attack via compromised vehicles (without RSU) occurs when malicious vehicles V3 and V4 intercept and echo legitimate V1–V2 HELLO messages, causing the controller to perceive false multi-hop paths. Although physical links remain unchanged, the echoed reports create non-existent logical paths. This deception leads to incorrect routing decisions without fabricating physical links (Fig. 3.11).

The attack proceeds as follows:

1. **Normal V2V Topology Discovery:** Vehicles periodically exchange topology HELLO messages over V2V wireless links to announce neighbor connectivity.
2. **Legitimate Link Establishment:** A legitimate wireless link exists between vehicles V1 and V2, while vehicles V3 and V4 are within overhearing range but are not part of the V1–V2 link. No direct links exist between V1–V3, V1–V4, V2–V3, or V2–V4.
3. **HELLO Message Interception (Echo Preparation):** HELLO messages exchanged between V1 and V2 are overheard and copied by vehicles V3 and V4, which act as echoing adversaries.
4. **Echoed Topology Observation Reports:** Vehicles V3 and V4 replay the observed HELLO information toward the controller, reporting the same legitimate link ($V1 \leftrightarrow V2$) as if it were observed through different intermediate nodes: Observed link: $V1 \leftrightarrow V2$, reported by V3; Observed link: $V1 \leftrightarrow V2$, reported by V4. These reports do not fabricate new links but duplicate an existing link across multiple observation paths.
5. **Multipath Inference at the Controller (Attack Effect):** The controller aggregates the echoed observations and incorrectly infers multiple distinct paths involving intermediate nodes V3 and V4. As a result, the controller assumes the existence of non-existent paths: Path 1: $V1 \rightarrow V2$ (real); Path 2: $V1 \rightarrow V3 \rightarrow V2$ (false); Path 3: $V1 \rightarrow V4 \rightarrow V2$ (false); Path 4: $V1 \rightarrow V3 \rightarrow V4 \rightarrow V2$ (false).
6. **Faulty Routing Decisions:** Based on the falsely inferred multipath topology, the controller installs incorrect routing rules that rely on non-existent paths, leading to packet loss, increased delay, and routing instability.

Multipath Echo Attack (Compromised Vehicles): An attack where malicious vehicles intercept and echo legitimate topology messages to the controller, inducing multiple non-existent logical paths and poisoned path computation without fabricating physical links.

ME Attack via Compromised RSU

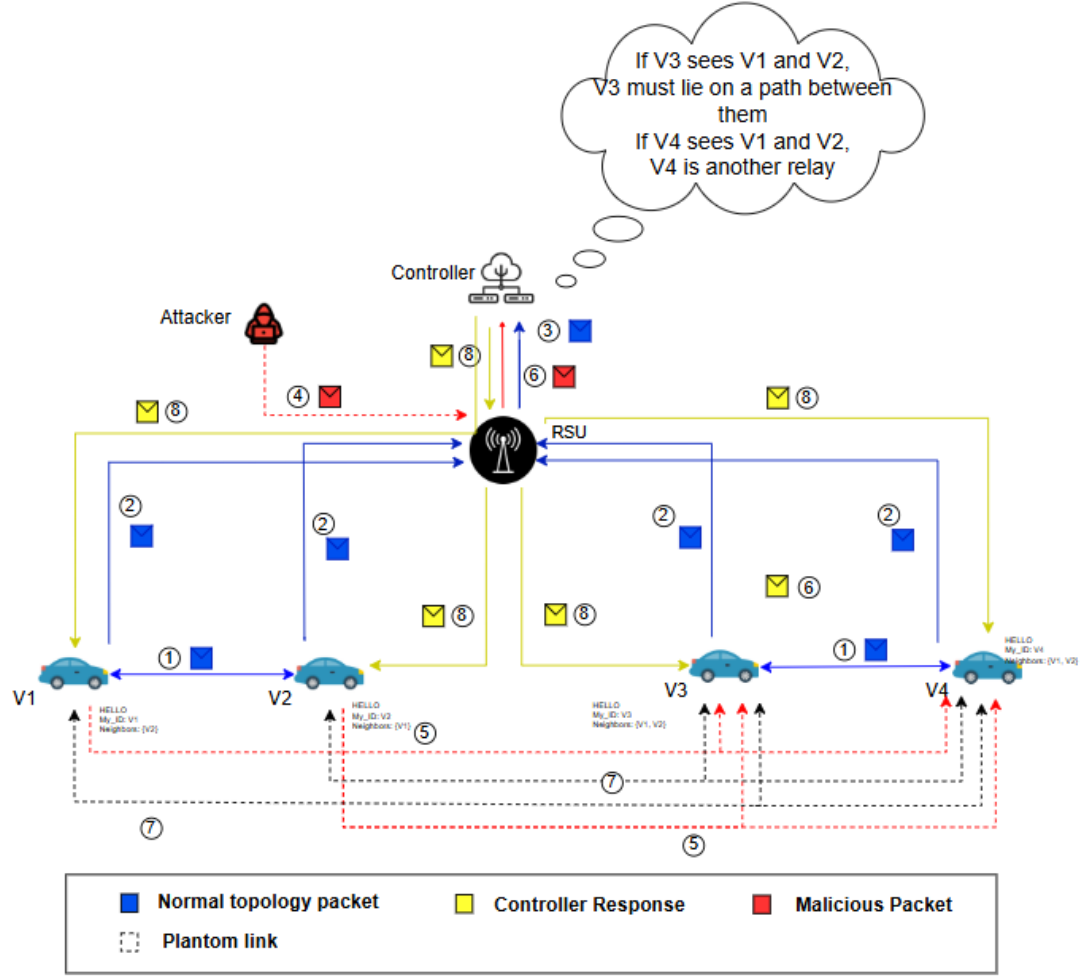


Figure 3.12: The RSU injects duplicated topology information related to V1–V2, creating false paths. (With RSU)

ME Attack Steps (RSU Aggregation Variant) The Multipath Echo (ME) attack via compromised RSU occurs when a malicious RSU injects echoed observations into aggregated topology reports from vehicles, causing the controller to perceive false multi-hop paths (Fig. 3.12). Although the physical links remain unchanged, the manipulated aggregation creates logical paths that do not exist in reality. This deception leads to incorrect routing decisions without fabricating physical links.

The attack proceeds as follows:

1. **Normal V2V Topology Discovery:** Vehicles exchange topology HELLO messages over V2V wireless links to advertise neighbor connectivity.

2. **Legitimate Link Establishment:** Legitimate wireless links exist between vehicles V1–V2 and V3–V4. Vehicles V3 and V4 are within overhearing range of the V1–V2 communication but do not have direct links with either V1 or V2.
3. **Normal Topology Reporting to the RSU:** Vehicles forward their legitimate topology observations to the RSU for aggregation.
4. **Echo Injection by the Malicious RSU:** The RSU, acting maliciously, injects forged HELLO observations that falsely indicate the V1–V2 HELLO messages were overheard by vehicles V3 and V4. These injected observations, illustrated as red dotted links in the figure, do not fabricate new links but instead duplicate an existing legitimate link across multiple reporting nodes.
5. **Observed HELLO Reports (Forged):** The forged topology observations indicate: Observed link: V1 $\leftrightarrow$ V2, reported by V3; Observed link: V1 $\leftrightarrow$ V2, reported by V4.
6. **Aggregated Topology Forwarding to the Controller:** The malicious RSU forwards an aggregated topology report to the controller containing both legitimate and injected echo observations: V3 reports observing the V1 $\leftrightarrow$ V2 link; V4 reports observing the V1 $\leftrightarrow$ V2 link.
7. **False Multipath Inference at the Controller:** Upon aggregating the echoed observations, the controller incorrectly infers the existence of multiple distinct paths involving vehicles V3 and V4: Path 1: V1 $\rightarrow$ V2 (real); Path 2: V1 $\rightarrow$ V3 $\rightarrow$ V2 (false); Path 3: V1 $\rightarrow$ V4 $\rightarrow$ V2 (false); Path 4: V1 $\rightarrow$ V3 $\rightarrow$ V4 $\rightarrow$ V2 (false).
8. **Faulty Routing Decisions:** Based on the falsely inferred multipath topology, the controller installs incorrect routing rules that rely on non-existent paths, leading to packet loss and routing instability.

Multipath Echo Attack (Compromised RSU): An attack where a malicious RSU injects echoed topology observations into aggregated vehicle reports, inducing multiple non-existent logical paths at the controller and poisoned path computation without fabricating physical links.

ME Attack via Compromised Controller

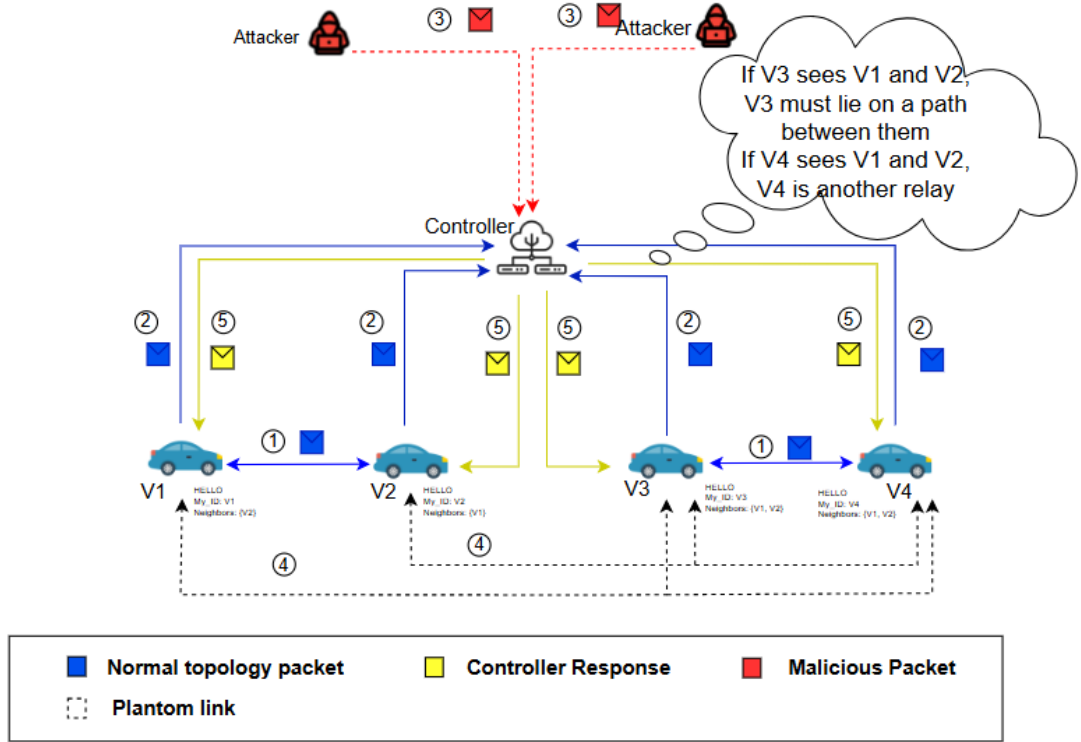


Figure 3.13: Controller-based Multipath Echo Attack (Without RSU): Controller internally duplicates V1–V2 link across multiple inferred paths.

ME Attack Steps (Vehicle-based: Direct Injection) The Multipath Echo (ME) attack via compromised controller (without RSU) occurs when a malicious controller directly injects echoed observations into its topology database from V2V communications, causing it to perceive false multi-hop paths (Fig. 3.14). Although the physical links remain unchanged, the manipulated internal topology creates logical paths that do not exist in reality. This deception leads to incorrect routing decisions without fabricating physical links.

The attack proceeds as follows:

1. **Normal V2V Topology Discovery:** Vehicles exchange topology HELLO messages over V2V wireless links to advertise neighbor connectivity.
2. **Legitimate Link Establishment:** Legitimate wireless links exist between vehicles V1–V2 and V3–V4. Vehicles V3 and V4 are within overhearing range of the V1–V2 communication but do not have direct links with either V1 or V2. Each vehicle independently reports its topology observations directly to the controller.

3. **Echo Injection by the Malicious Controller:** The controller, acting maliciously, injects forged HELLO observations into its internal topology database, making it appear as if the legitimate V1–V2 HELLO messages were overheard and relayed through vehicles V3 and V4. These injected observations duplicate an existing legitimate link across multiple reporting paths. Echoed topology observations: Observed link: $V1 \leftrightarrow V2$, relayed via V3; Observed link: $V1 \leftrightarrow V2$, relayed via V4.
4. **False Multipath Inference:** Based on the duplicated observations, the controller incorrectly infers the existence of multiple paths involving vehicles V3 and V4: Path 1: $V1 \rightarrow V2$ (real); Path 2: $V1 \rightarrow V3 \rightarrow V2$ (false); Path 3: $V1 \rightarrow V4 \rightarrow V2$ (false); Path 4: $V1 \rightarrow V3 \rightarrow V4 \rightarrow V2$ (false).
5. **Faulty Routing Decisions:** The controller installs routing policies based on the falsely inferred multipath topology, resulting in incorrect packet forwarding and routing instability.

Multipath Echo Attack (No RSU): An attack where a compromised controller injects echoed topology observations directly into its database from V2V reports, inducing multiple non-existent logical paths and poisoned path computation without fabricating physical links.

ME Attack Steps via Compromised Controller (with RSU)

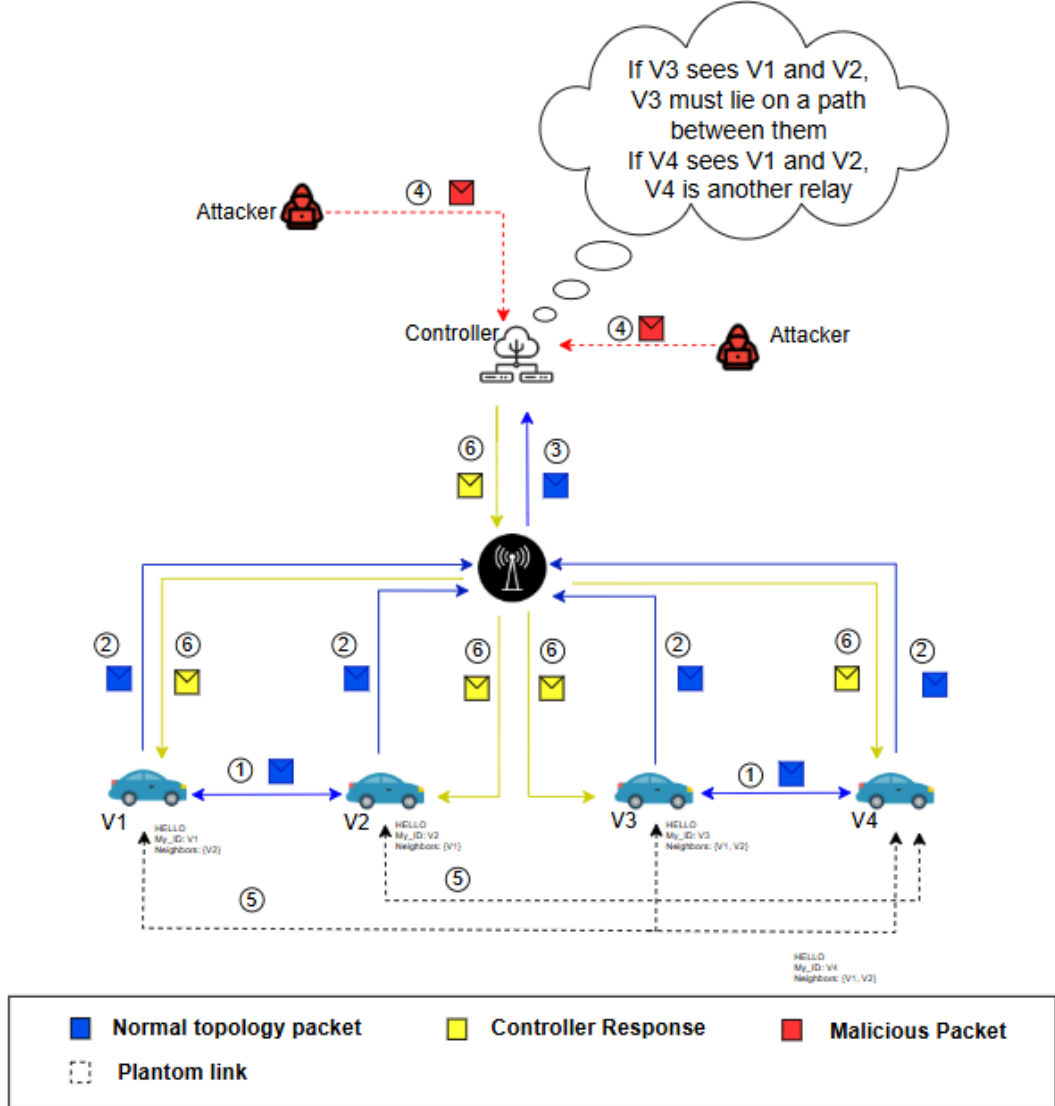


Figure 3.14: Controller-based Multipath Echo Attack (With RSU): The controller internally duplicates the V1–V2 link across multiple inferred paths.

The Multipath Echo (ME) attack occurs when a malicious controller, receiving topology information aggregated by RSUs, injects echoed observations into otherwise legitimate topology reports, causing itself to perceive false multi-hop paths (Fig. 3.14). Although the physical links remain unchanged, the manipulated reports create logical paths that do not exist in reality. This deception leads to incorrect routing decisions without fabricating physical links.

The attack proceeds as follows:

- 1. Normal V2V Topology Discovery:** Vehicles exchange topology HELLO messages over V2V wireless links to advertise neighbor connectivity.

2. **Legitimate Link Establishment:** Legitimate wireless links exist between vehicles V1–V2 and V3–V4. Vehicles V3 and V4 are within overhearing range of the V1–V2 communication but do not have direct links with either V1 or V2. Each vehicle forwards both its own and observed topology information to the RSU.
3. **Topology Reporting via RSU:** Vehicles forward their normal topology observations to the RSU. The RSU aggregates the received topology reports and forwards the aggregated topology information to the controller.
4. **Echo Injection by the Malicious Controller:** The controller injects forged HELLO observations into the aggregated topology information, making it appear as if the legitimate V1–V2 HELLO messages were overheard and relayed through vehicles V3 and V4. These injected observations duplicate an existing legitimate link across multiple reporting paths.
 Echoed topology observations: Observed link: $V1 \leftrightarrow V2$, relayed via V3;
 Observed link: $V1 \leftrightarrow V2$, relayed via V4.
5. **False Multipath Inference:** Based on the duplicated observations, the controller incorrectly infers the existence of multiple paths involving vehicles V3 and V4: Path 1: $V1 \rightarrow V2$ (real); Path 2: $V1 \rightarrow V3 \rightarrow V2$ (false); Path 3: $V1 \rightarrow V4 \rightarrow V2$ (false); Path 4: $V1 \rightarrow V3 \rightarrow V4 \rightarrow V2$ (false).
6. **Faulty Routing Decisions:** The controller installs routing policies based on the falsely inferred multipath topology, resulting in incorrect packet forwarding and routing instability.

Multipath Echo Attack: An attack in which legitimate topology messages are replayed or misinterpreted to induce multiple non-existent logical paths, resulting in poisoned controller path computation without fabricating physical links.

3.4.4 Attack Signature Characterization

Effective detection of temporal-echo attacks requires that observable, measurable behavioral conditions be formally derived from the mathematical definition of each attack variant. We define an *attack signature* as a necessary and sufficient logical condition on control-plane beacon or heartbeat traffic that is triggered if and only if the corresponding attack is in progress. Nine signatures are derived across the three attack variants, forming the theoretical foundation of both the lightweight and full-stack detection components. The complete attack divergence model is given in eq:attack_divergence.

Let $\tau_r^{(k)}$ denote the controller reception time of the k -th beacon, $\tau_s^{(k)}$ the embedded sender timestamp, $T_b = 100$ ms the nominal beacon interval per IEEE 802.11p, and ϵ the maximum tolerable one-way propagation delay. Let $\mathcal{R}(e_{ij}, t)$ denote the set of nodes reporting link e_{ij} at time t , $\mathcal{P}(V_i, V_j)$ the set of controller-inferred paths between V_i and V_j , r_{comm} the maximum V2V communication range, and W the heartbeat observation window.

Mobility-Induced Attack Amplification in SDVNs

A defining characteristic of SDVNs that distinguishes temporal-echo attacks from analogous attacks in wired SDN is the role of vehicle mobility in creating exploitable ambiguity windows. In wired SDN, topology changes are infrequent and structurally suspicious; any unexplained link event warrants immediate scrutiny. In SDVNs, link formation and breakdown are continuous, expected, and legitimate consequences of vehicle movement. The controller must therefore tolerate a degree of temporal uncertainty in its topology view, and it is precisely this necessary tolerance that temporal-echo attacks exploit.

Link Lifetime Model. Consider two vehicles V_i and V_j moving with relative velocity v_{rel} (m/s). The minimum link lifetime — the shortest duration for which the link e_{ij} can remain active — occurs in the worst-case head-on geometry and is bounded by:

$$L_{\text{link}} = \frac{2 r_{\text{comm}}}{v_{\text{rel}}} \quad (3.30)$$

At urban speeds ($v_{\text{rel}} \approx 14$ m/s, i.e., 50 km/h) with $r_{\text{comm}} = 300$ m, $L_{\text{link}} \approx 43$ s. At highway speeds ($v_{\text{rel}} \approx 67$ m/s, i.e., 120 km/h relative), $L_{\text{link}} \approx 9$ s. The number of beacons exchanged during a single link lifetime is therefore:

$$N_{\text{beacon}} = \left\lfloor \frac{L_{\text{link}}}{T_b} \right\rfloor \quad (3.31)$$

ranging from approximately 90 beacons at highway speeds to 430 beacons at urban speeds. This short link lifetime means the controller must continuously accept new link formations and terminations as normal events, which is the structural condition that temporal-echo attacks exploit.

Topology Convergence Delay. After a physical link e_{ij} breaks at time t_b , the controller does not receive notification until the next beacon cycle completes.

The maximum duration during which the controller’s topology G_t^C diverges from the real topology G_t^R due to normal mobility — the *topology convergence delay* — is bounded by:

$$\tau_{\text{conv}} \leq T_b + \tau_{\text{prop}} \quad (3.32)$$

where τ_{prop} is the message propagation delay from the vehicle to the controller via RSU. During this window τ_{conv} , it is *legitimate* for the controller to hold topology information about a link that no longer physically exists. This window is the primary exploitation surface of the TTW attack: a replayed beacon injected within τ_{conv} of a link breakdown is indistinguishable from a legitimately delayed topology update, because both arrive within the normal convergence tolerance.

RSU Handover Window. When vehicle V_i transitions from the coverage zone of RSU_a to that of RSU_b , there exists a handover window W_{ho} during which neither RSU has fresh information about V_i :

$$W_{\text{ho}} \approx \frac{r_{\text{overlap}}}{v_i} \quad (3.33)$$

where r_{overlap} is the overlap distance between adjacent RSU coverage zones and v_i is vehicle speed. During W_{ho} , a liveness gap for V_i is legitimate, expected, and accepted by the controller. The BSHH attacker exploits this window by replaying stale heartbeats that arrive precisely during the handover gap, making the attack indistinguishable from the natural liveness interruption that the controller expects during coverage transitions.

Dynamic Observer Population. The set of vehicles that can legitimately observe a given link e_{ij} at time t — denoted $\mathcal{R}^*(e_{ij}, t)$ — is not static. It changes continuously as vehicles enter and leave the communication range of V_i and V_j . The expected size of this set under a vehicular density $\lambda(t)$ (vehicles per unit road length, time-varying) and communication range r_{comm} is:

$$E[|\mathcal{R}^*(e_{ij}, t)|] = 2r_{\text{comm}} \cdot \lambda(t) \quad (3.34)$$

In high-density urban scenarios ($\lambda(t)$ large), many vehicles legitimately observe the same link, making it harder to distinguish genuine from echo reports — an effect that amplifies the ME attack under high vehicle density. In low-density highway scenarios ($\lambda(t)$ small), even a single echo reporter significantly inflates the observer count, making the attack easier to detect via ME-S1. The dynamic nature of $\lambda(t)$ means that a static ρ_{max} bound in ME-S1 would produce unacceptable false-positive rates in dense urban conditions and miss attacks in sparse highway conditions. ME-S1 must therefore use a mobility-aware dynamic bound, as defined in eq:me_s1_dynamicbelow.

These three mobility-induced conditions — topology convergence delay, RSU handover window, and dynamic observer population — collectively explain why temporal-echo attacks are *SDVN-specific threats that cannot arise in wired SDN*: in wired SDN, links do not break spontaneously due to physical movement, liveness gaps do not occur during infrastructure handovers, and observer populations do not fluctuate with vehicle density. The attack signatures defined in the following subsections are therefore formulated with explicit reference to these mobility parameters.

TTW Attack Signatures

The Topology Time-Warp attack corrupts the controller’s temporal perception of the network by replaying or reordering legitimate beacon messages with forged timestamps. Three signatures characterize its observable effects.

TTW-S1 (Timestamp Staleness) detects beacons whose embedded sender timestamp is inconsistent with their reception time, indicating that a stored beacon is being replayed with a freshness-offset exceeding the propagation bound:

$$\Delta_k = \tau_r^{(k)} - \tau_s^{(k)} > T_b + \epsilon \quad (3.35)$$

TTW-S2 (Sequence Inversion) detects the condition in which a later-received beacon from the same source carries an earlier sender timestamp than a previously received beacon, indicating that an older message has been replayed out of sequence:

$$\exists j < k : \tau_s^{(j)} > \tau_s^{(k)} \wedge \tau_r^{(j)} < \tau_r^{(k)} \quad (3.36)$$

TTW-S3 (Cross-Reporter Inconsistency) detects the condition in which two distinct reporters of the same link e_{ij} carry timestamps that differ by more than one beacon interval, indicating that at least one of the reports is a replay of a stale observation:

$$\exists V_a \neq V_b : \left| \tau_s^{V_a \rightarrow C}(e_{ij}) - \tau_s^{V_b \rightarrow C}(e_{ij}) \right| > T_b \quad (3.37)$$

Mobility Amplification of TTW. The TTW attack is specifically amplified by the topology convergence delay defined in eq:convergence_delay. *At highway speeds, a link between two vehicles breaks and reforms after the physical link has broken, because this delay is indistinguishable from normal propagation latency.* The TTW attacker injects a replayed beacon with a forged timestamp during exactly this window, exploiting the controller’s necessary tolerance for convergence delay. The higher the vehicle speed, the shorter L_{link} , the more frequently links break legitimately, and the more credible a replayed beacon appears — because the controller has recently observed the link and cannot rule out a delayed update. Signatures TTW-S1 and TTW-S2 are calibrated to the beacon interval T_b , which is the minimum observable unit of timing inconsistency; both are therefore mobility-speed agnostic in their threshold but the detection probability increases as v_{rel} increases because faster speeds produce larger timestamp discrepancies for the same replay delay. TTW-S3 is mobility-aware through T_b : at the nominal 100 ms beacon interval, a cross-reporter timestamp gap exceeding T_b is anomalous regardless of vehicle speed, because all legitimate reporters of the same link observe it within the same beacon cycle.

BSHH Attack Signatures

The Beacon State Heartbeat Hijack attack corrupts the controller’s liveness model by replaying legitimate heartbeat messages under a spoofed sender identity. Three

signatures characterize its observable effects, where H^{V_a} denotes the heartbeat message received from physical sender V_a .

BSHH-S1 (Duplicate Liveness Assertion) detects the condition in which two distinct physical senders submit heartbeats claiming to originate from the same node V_k within the same observation window, a structural impossibility in an uncompromised network:

$$\exists V_a \neq V_b : \text{Sender}(H^{V_a}) = \text{Sender}(H^{V_b}) = V_k, \quad \tau_r^{V_a} \approx \tau_r^{V_b} \quad (3.38)$$

BSHH-S2 (Timestamp Regression) detects the condition in which the heartbeat timestamp of a given node decreases between consecutive controller-received messages, indicating that an older heartbeat has been replayed to maintain a false liveness signal:

$$\tau_s^{(k+1)}(V_i) < \tau_s^{(k)}(V_i) \quad (3.39)$$

BSHH-S3 (Liveness-Beacon Inconsistency) detects the condition in which a heartbeat asserts the liveness of node V_i during a window W in which no beacon was received from V_i , indicating that the heartbeat originates from an impersonating node rather than from V_i itself:

$$\text{Heartbeat}(V_i, t) = \text{TRUE} \wedge \text{Beacon}(V_i, [t - W, t]) \quad (3.40)$$

Mobility Amplification of BSHH. The BSHH attack exploits the RSU handover window defined in eq:handover_window. When vehicle V_i transitions between RSU coverage zones, the controller legitimately expects a liveness gap of duration W_{ho} during which no fresh heartbeat from V_i is received. The BSHH attacker times the replay of stale heartbeats to coincide with this gap, making the replayed liveness signal appear as the first post-handover heartbeat from V_i . This is particularly effective at high vehicle speeds, where handover events are frequent and W_{ho} is short but non-zero. Signature BSHH-S3 addresses this by checking for liveness assertions that are not accompanied by any beacon within window W . The window W must be set relative to the expected handover duration: specifically, $W > W_{\text{ho}}$ to avoid triggering false positives during legitimate handovers, while being small enough to detect BSHH injections that persist beyond the handover window. In practice, W is set to $2 \cdot W_{\text{ho}}$ based on the RSU deployment geometry and the prevailing vehicle speed, making BSHH-S3 a mobility-calibrated signature whose threshold adapts to the infrastructure topology. Signature BSHH-S1 is mobility-independent — duplicate liveness assertions from distinct physical senders cannot be explained by mobility under any speed regime — and therefore provides a clean detection signal even when BSHH-S2 and BSHH-S3 are subject to mobility-induced false-positive pressure.

ME Attack Signatures

The Multipath Echo attack exploits the controller’s path inference logic by causing legitimate topology observations to be reported by physically inconsistent witnesses, inducing false path diversity without fabricating any individual message.

The ME attack represents a *false witness attestation* problem rather than a content manipulation problem: the attacker does not forge what was communicated, but falsely claims from where it was observed. Three signatures characterize its observable effects.

ME-S1 (Dynamic Reporter Count Excess): The number of distinct nodes reporting link e_{ij} at time t exceeds the mobility-aware maximum plausible observer count. Unlike a static bound, the threshold is computed from the instantaneous vehicle density estimate $\hat{\lambda}(t)$, which is updated from RSU-observed beacon rates at each time step:

$$|\mathcal{R}(e_{ij}, t)| > \rho_{\max}(\hat{\lambda}(t), r_{\text{comm}}) \quad (3.41)$$

where the dynamic bound is:

$$\rho_{\max}(\hat{\lambda}(t), r_{\text{comm}}) = \lceil (1 + \mu) \cdot 2 r_{\text{comm}} \cdot \hat{\lambda}(t) \rceil \quad (3.42)$$

and $\mu > 0$ is a tolerance margin (recommended $\mu = 0.3$) that accommodates natural density fluctuation without generating false positives. The density estimate $\hat{\lambda}(t)$ is maintained by the RSU from the count of distinct vehicle beacons received per unit road length within the current beacon window:

$$\hat{\lambda}(t) = \frac{|\mathcal{V}_{\text{active}}(t)|}{2 r_{\text{comm}}} \quad (3.43)$$

where $\mathcal{V}_{\text{active}}(t)$ is the set of vehicles from which at least one beacon was received within the current sliding window $[t - T_b, t]$. Using a dynamic bound makes ME-S1 robust across the full range of SDVN deployment scenarios: in high-density urban environments, the bound rises proportionally with density, preventing false positives; in low-density highway environments, the bound falls, increasing sensitivity to even single-vehicle echo injections.

$$|\mathcal{R}(e_{ij}, t)| > \rho_{\max}(\lambda, r_{\text{comm}}) \quad (3.44)$$

ME-S2 (Path Diversity Inflation) detects the condition in which the number of controller-inferred paths between two nodes increases by more than the mobility-consistent bound $\delta_{\max}$ within a single beacon interval, indicating that echo-induced phantom paths have been inserted into the controller's topology graph:

$$|\mathcal{P}(V_i, V_j)|_t - |\mathcal{P}(V_i, V_j)|_{t-1} > \delta_{\max} \quad (3.45)$$

ME-S3 (Reporter-Range and Signal Inconsistency) detects the condition in which a reporter V_k claims to have observed link e_{ij} but either its GPS-attested position places it beyond communication range of both endpoints, or its reported received signal strength is below the minimum consistent with direct-range reception. Unlike ME-S1 and ME-S2, which are statistical signatures, ME-S3 is a per-reporter physical consistency check that directly identifies the individual false witness:

$$V_k \in \mathcal{R}(e_{ij}, t) \wedge \left(d(\text{pos}_{V_k}, e_{ij}) > r_{\text{comm}} \vee \text{RSSI}_{V_k \leftarrow V_i} < \text{RSSI}_{\min}(r_{\text{comm}}) \right) \quad (3.46)$$

where $d(\text{pos}_{V_k}, e_{ij})$ is the Euclidean distance from the GPS-attested position of V_k to the nearest endpoint of link e_{ij} , and $\text{RSSI}_{\min}(r_{\text{comm}})$ is the minimum expected received signal strength at the maximum communication range under the adopted path-loss model. It is important to note that ME is not solvable by content-only cryptographic schemes such as broadcast encryption, since ME attackers are authorized network participants who legitimately overhear wireless transmissions. ME-S3 addresses this by binding topology reports to physically verifiable location and signal evidence, as elaborated in Section 3.4.6.

Mobility Amplification of ME. The ME attack is amplified by two distinct mobility-induced effects. First, in high-density urban environments, the dynamic observer population model eq:expected_observers shows that many vehicles legitimately observe the S1. The dynamic bound in eq:me_s1_dynamic directly counteracts this by scaling the detection threshold in mobility environments, delayed topology reports from vehicles that recently moved out of range are accepted. That was within range of V_1 and V_2 at time $t - \Delta$ may legitimately submit a topology observation with a timestamp of $t - \Delta$ at a later time due to RSU aggregation and propagation delays. The ME attacker exploits this by injecting echo reports timed to coincide with the expected delay distribution of legitimate late-arriving reports. ME-S3 directly addresses this by checking the *current* GPS-attested position of the reporter, not its historical position — even if V_3 was legitimately in range at $t - \Delta$, the location-binding signature eq:loc_binding_s3 binds the report to the position at the time of signing, not at the time of reception. A reporter that has since moved out of range fails the range consistency check in eq:me_a_ccept_s3 regardless of its historical proximity, closing the mobility-induced delay window that the ME attacker exploits.

Table ?? summarizes all nine signatures, their observable features, and the attacker positions against which each is effective.

3.4.5 Detection Logic

Detection logic is provided in two versions. The lightweight version (**LW**) applies rule-based evaluation of the nine signatures defined in Section ?? and is suitable for deployment on resource-constrained OBUs and RSUs. The full-stack version (**FS**) applies a Temporal Graph Neural Network that models the sequential evolution of the beacon-based SDVN topology graph under vehicular mobility, providing superior detection performance against adaptive adversaries that may evade individual signature conditions.

Lightweight Rule-Based Detection

The lightweight detector maintains a sliding window $\mathcal{W}$ of the most recent $W_{\max}$ topology events and evaluates all nine signatures against each incoming event e . For each signature Sig_k , a binary indicator $f_k \in \{0, 1\}$ is computed. A weighted anomaly score is then computed as:

$$s(e) = \sum_{k=1}^9 w_k \cdot \mathbf{1}[\text{Sig}_k \text{ triggered}] \quad (3.47)$$

where $w_k \geq 0$ is the weight assigned to signature Sig_k , with $\sum_{k=1}^9 w_k = 1$. Weights are assigned proportionally to the empirical false-positive rate of each signature on benign traffic in the NS-3/SUMO simulation, such that signatures with lower false-positive rates receive higher weights. A detection alert of variant α is raised when $s(e) > \theta_{\text{LW}}$, where θ_{LW} is determined by maximizing the F_1 score on the validation split of the simulation dataset. The full procedure is given in Algorithm 1.

The per-event time complexity of the lightweight detector over a window of $|\mathcal{W}|$ events and $|\mathcal{S}| = 9$ fixed signatures is:

$$T_{\text{LW}}(|\mathcal{W}|, |\mathcal{S}|) = O(|\mathcal{W}| \cdot |\mathcal{S}|) \quad (3.48)$$

Since $|\mathcal{S}| = 9$ is a constant determined by the attack model, this reduces to:

$$T_{\text{LW}}(|\mathcal{W}|) = O(|\mathcal{W}|) \quad (3.49)$$

confirming that detection overhead scales linearly with window size only and is independent of network scale, which is essential for OBU and RSU deployability within the 100 ms SDVN latency budget.

Full-Stack Temporal GNN Detection

The full-stack detector models the SDVN topology as a sequence of graph snapshots, one per beacon interval. At time t , the topology graph snapshot is defined as:

$$\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, \mathbf{X}_t, \mathbf{A}_t) \quad (3.50)$$

where $\mathcal{V}_t$ is the set of active vehicles and RSUs, $\mathcal{E}_t$ is the set of reported topology links, $\mathbf{A}_t \in R^{|\mathcal{V}_t| \times |\mathcal{V}_t|}$ is the weighted adjacency matrix whose entry $\mathbf{A}_{t,uv}$ encodes the normalized timestamp freshness of the link report between u and v , and $\mathbf{X}_t \in R^{|\mathcal{V}_t| \times d}$ is the node feature matrix. The feature vector of each node v is:

$$\mathbf{x}_v = [\text{id}_v, \tau_s^{(v)}, c_v^W, \Delta s_v, \rho_v] \in R^d \quad (3.51)$$

where id_v is the node identifier embedding, $\tau_s^{(v)}$ is the last received beacon timestamp, c_v^W is the beacon count within sliding window W , Δs_v is the sequence number gap between consecutive received beacons, and ρ_v is the reporter count for links adjacent to v . The features Δs_v and ρ_v directly encode the TTW and ME anomaly signals, respectively, allowing the TGN to learn attack-specific graph evolution patterns from labeled simulation data

Edge Weight Computation. The adjacency matrix entry $\mathbf{A}_{uv}(t)$ encodes the freshness of the most recent topology report for link (u, v) using an exponential decay function:

$$\mathbf{A}_{uv}(t) = \exp\left(-\frac{\tau_r^{(t)} - \tau_s^{(uv)}}{\gamma \cdot T_b}\right) \quad (3.52)$$

where $\tau_s^{(uv)}$ is the most recent sender timestamp on the link report between u and v , and $\gamma > 0$ is a decay rate hyperparameter tuned on validation data. This formulation assigns unit weight to perfectly fresh reports and approaches zero for

stale reports, directly penalizing TTW-injected stale-timestamped edges during neighbourhood aggregation without requiring explicit threshold tuning.

Online Operation Mode. The TGN operates in online mode, updating node memories and computing anomaly scores at each incoming topology event rather than at the end of each beacon interval. This is essential because a single anomalous beacon injected mid-interval would be masked by averaging in batch operation. Each topology event e received by the SDN controller immediately triggers a memory update and anomaly scoring pass, with the persistent memory state $\mathbf{m}_v(t)$ maintained incrementally across events.

New Node Initialization. When a vehicle V_{new} appears for the first time — a frequent occurrence in the high-mobility SDVN context — its initial embedding is set to:

$$\mathbf{h}_{V_{\text{new}}}(t^-) = \mathbf{0} \in R^d \quad (3.53)$$

This zero initialization ensures that a newly appearing node contributes no spurious prior embedding signal, while still allowing immediate anomaly detection from its first event if it triggers any of the nine attack signatures or produces an anomalous topology observation pattern.

Mobility-Aware TGN Design. Three TGN design decisions are specifically motivated by the SDVN mobility model established in Section ??.

First, the time-elapsed encoding $\phi(\Delta t_v) = \log(1 + \Delta t_v/T_b)$ captures the expected inter-event gap under normal vehicular mobility. A gap of exactly T_b (one beacon interval) encodes as $\phi = \log 2 \approx 0.69$, representing normal operation. A gap of L_{link} eq:link lifetime without a subsequent beacon encodes as $\phi = \log(1 + L_{\text{link}}/T_b)$, signaling a mobility-consistent link termination. A gap that significantly exceeds L_{link} encodes as a large ϕ value that the GRU memory update learns to associate with anomalous temporal patterns, since no legitimate mobile link should produce such gaps under the prevailing speed regime.

Second, the freshness-decayed edge weight $\mathbf{A}_{uv}(t)$ eq:edge weight is calibrated by decay rate γ , which is set based on the expected link lifetime at the prevailing vehicle speed. Specifically, γ is chosen such that $\mathbf{A}_{uv}(t) \approx 0.5$ when $\tau_r^{(t)} - \tau_s^{(uv)} = L_{\text{link}}/2$ — the halfway point of expected link lifetime — ensuring that edges approaching natural expiry are already downweighted before they are expected to break, while edges significantly past their expected lifetime (TTW-replayed stale edges) receive near-zero weight.

Third, the sliding window size W_{max} of the lightweight detector is set to retain events from the most recent $\lceil L_{\text{link}}/T_b \rceil$ beacon intervals, ensuring that the window spans at least one full link lifetime under the prevailing speed. This prevents the window from being too short (missing cross-interval TTW sequence inversions detectable by TTW-S2) or too long (retaining events from vehicles that have long since departed, producing false positive TTW-S3 detections from legitimately stale cross-reporter timestamps). The window size is therefore a mobility-adaptive parameter that should be updated when the estimated vehicle speed changes significantly, as detected from the rate of RSU handover events.

Temporal Memory Update. Unlike static GNN methods, the Temporal Graph Network maintains a persistent memory state for each node that captures its behavioral history across beacon intervals. At each event, the memory of node v is updated using a Gated Recurrent Unit:

$$\mathbf{m}_v(t) = \text{GRU}(\mathbf{h}_v(t^-), \mathbf{x}_v(t), \phi(\Delta t_v)) \quad (3.54)$$

where $\mathbf{h}_v(t^-)$ is the node embedding from the immediately preceding beacon event, $\mathbf{x}_v(t)$ is the current feature vector from eq:node_features, and $\phi(\Delta t_v) = \log(1 + \Delta t_v/T_b)$ is a time-elapsd encoding that scales the inter-event gap by the beacon interval T_b to account for vehicular mobility dynamics. This encoding ensures that nodes that have been silent for an unusually long interval receive a proportionally amplified temporal signal, which is directly relevant to the liveness spoofing detection in BSHH-S3.

Message Passing. Node embeddings are refined through L rounds of neighbourhood aggregation over the topology graph:

$$\mathbf{h}_v^{(l+1)} = \sigma(\mathbf{W}^{(l)} \cdot \text{MEAN}(\{\mathbf{h}_u^{(l)} \odot \mathbf{A}_{uv}(t) : u \in \mathcal{N}(v, t)\}) + \mathbf{b}^{(l)}) \quad (3.55)$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are trainable weight matrix and bias at layer l , $\sigma(\cdot)$ denotes the ReLU activation function, $\odot$ is element-wise multiplication, and $\mathcal{N}(v, t)$ is the set of topological neighbours of v in $\mathcal{G}_t$. The edge weight $\mathbf{A}_{uv}(t)$ modulates the contribution of each neighbour by the freshness of the link report, causing stale-timestamped edges — characteristic of TTW attacks — to receive lower aggregation weights and propagate weaker embedding signals to the central node.

Anomaly Scoring. The anomaly score for node v at time t is computed from the final-layer embedding:

$$\hat{y}_v(t) = \sigma(\mathbf{w}^\top \mathbf{h}_v^{(L)}(t)) \quad (3.56)$$

where $\mathbf{w} \in R^d$ is a trainable scoring vector and $\sigma(\cdot)$ here denotes the sigmoid function mapping the score to the interval $(0, 1)$. A full-stack detection alert for node v is raised when $\hat{y}_v(t) > \theta_{\text{FS}}$, where θ_{FS} is selected by maximizing the Matthews Correlation Coefficient (MCC) on held-out validation data to ensure robust performance under class imbalance between benign and attack events. The complete inference procedure is given in Algorithm 3.

The TGN is trained on labeled topology event traces generated by the NS-3/SUMO simulation framework under all twelve attack scenarios (three variants $\times$ four attacker positions). A standard 70/15/15 train/validation/ test split is used. The model is trained using binary cross-entropy loss with class-weight balancing to account for the natural imbalance between benign and attack events in realistic vehicular traffic traces.

3.4.6 Cryptographic Placement Analysis

A fundamental design question for any hybrid detection-plus-mitigation framework is whether cryptographic enforcement should be applied *before* detection (as a

pre-detection filter that eliminates certain attack vectors before they reach the detection engine) or *after* detection (as a post-detection response that mitigates confirmed attacks). Prior works apply cryptography uniformly at the message layer without resolving this question, resulting in architectures that either apply expensive cryptographic overhead where it provides no benefit, or fail to prevent attacks where it could have.

This section formally analyzes, for each combination of attack variant and attacker position, whether the available cryptographic primitives (HMAC-timestamp-nonce, threshold aggregate signatures, and Kyber+Saber PQC key agreement) eliminate or merely reduce the attack surface. The analysis yields a principled architectural placement rule that governs the TETA-Guard framework design.

Placement Principle. A cryptographic primitive provides *pre-detection value* for a given attack vector if and only if its verification condition is necessary for the attack to succeed — that is, the attack cannot be executed by a node that passes the cryptographic check. If a node can execute the attack while holding a valid key and producing a valid signature (i.e., the attacker is an authorized participant), then cryptography provides no pre-detection value for that vector and must be applied post-detection.

TTW Analysis. For the vehicle-origin and RSU-origin TTW vectors, the attacker replays a previously legitimate beacon with a forged timestamp. Applying HMAC-timestamp-nonce verification $\text{eq:hmac}_{\text{verify}} - \text{eq:nonce as a pre-detection filter prevents this replay because the nonce has already been consumed and the freshness condition cryptographic filtering for these vectors. For the controller-origin TTW vector, the malicious detection value; TGN-based detection is the primary defense, and blockchain-enforced isolation}$

BSHH Analysis. For the vehicle-origin and RSU-origin BSHH vectors, the attacker impersonates another node's identity in the heartbeat sender field. Threshold aggregate signature verification $\text{eq:threshold}_{\text{sig}}$ prevents this impersonation because a node cannot $\text{detection cryptographic filtering for these vectors. For the controller-origin BSHH vector, the malicious layer authentication applies, and TGN-based detection with post-detection smart contract isolation}$

ME Analysis. The ME attack is qualitatively different from TTW and BSHH because the attacker does not forge content or identities — it falsely *witnesses* a legitimate communication from a physically inconsistent position. Standard content-authenticated schemes (HMAC, aggregate signatures) provide no pre-detection value because the attacker produces correctly signed reports; the attack evades message-layer authentication entirely. The correct cryptographic primitive is *location-binding signatures* that bind each topology observation to the reporter's GPS-attested position and RSSI measurement, as defined in $\text{eq:loc}_{\text{binding}}_{\text{msg}} - \text{eq:loc}_{\text{binding}}_{\text{sig}}$. These provide pre-detection value for vehicle-origin and RSU-origin ME vectors where position evidence can be externally verified. For controller-origin ME, the controller injects false observations directly into its topology database; no external $\text{detection cryptographic filter applies.}$

The complete placement analysis is summarized in Table ??, which maps each combination of attack variant and attacker position to the cryptographic primitive, its placement, and whether it eliminates or reduces the attack surface.

3.4.7 Mitigation Algorithms

Mitigation logic is provided in two versions consistent with the dual-mode framework. The lightweight mitigation (**LW**) employs HMAC-timestamp-nonce verification as a pre-detection filter for external attack vectors, imposing negligible computational overhead. The full-stack mitigation (**FS**) employs lattice-based threshold aggregate signatures over Kyber+Saber-established session keys for pre-detection authentication, location-binding signatures for ME-specific false witness prevention, and Hyperledger Fabric smart contract-driven automated isolation and routing correction as post-detection response. Detection and mitigation logic are strictly separated: detection produces an anomaly score or alert; mitigation acts on confirmed alerts only.

Lightweight Mitigation: HMAC-Timestamp-Nonce Verification

Each beacon or topology update message m from node V_i at sender timestamp τ_s is constructed as:

$$m' = m \parallel \tau_s \parallel \text{nonce}_i \quad (3.57)$$

and authenticated using a session key $K_{V_i, \text{RSU}}$ established via the Kyber+Saber Hybrid-KEM [?]:

$$\text{MAC}_i = \text{HMAC-SHA256}(K_{V_i, \text{RSU}}, m') \quad (3.58)$$

The verifier (RSU or controller) accepts the message if and only if all three conditions hold simultaneously. First, the MAC must be valid:

$$\text{HMAC-SHA256}(K_{V_i, \text{RSU}}, m') = \text{MAC}_i^{\text{received}} \quad (3.59)$$

Second, the message must be temporally fresh:

$$|\tau_r - \tau_s| \leq T_b + \epsilon \quad (3.60)$$

Third, the nonce must not have been previously observed:

$$\text{nonce}_i \notin \mathcal{N}_{\text{seen}} \quad (3.61)$$

where $\mathcal{N}_{\text{seen}}$ is a per-sender nonce cache maintained at the verifier. Messages failing any condition are silently dropped before reaching the signature detector or TGN inference engine, eliminating the TTW and BSHH external attack vectors at the filter layer. The per-message overhead of HMAC-SHA256 is approximately 1–2 μs on ARM Cortex-M class OBU hardware [?], which is less than 2% of the 100 ms beacon interval and confirms lightweight deployability. Session key establishment uses the Kyber+Saber Hybrid-KEM, which achieves millisecond-level handshake latency compliant with the 10 ms V2X timing constraint [?]. The complete lightweight mitigation procedure is given in Algorithm 2.

Full-Stack Mitigation: Threshold Aggregate Signatures, Location-Binding Signatures, and Smart Contract Enforcement

Threshold Aggregate Signature Verification. For RSU-aggregated topology reports, each vehicle V_i signs its individual beacon observation under its private key. The RSU collects n such signatures and constructs a threshold aggregate valid only if at least $t \geq \lfloor n/2 \rfloor + 1$ individual signatures are verified:

$$\text{Verify}(\sigma_{\text{agg}}, \text{PK}_{\text{agg}}) = 1 \iff |\{i : \text{Verify}(\sigma_i, \text{msg}_i, \text{PK}_{V_i}) = 1\}| \geq t \quad (3.62)$$

The threshold aggregate signature is constructed using the lattice-based scheme of [?], which provides post-quantum security under the NTRU hardness assumption. Per-verification latency is approximately 1.6 ms [?], which is within the SDVN latency budget. This construction eliminates vehicle-origin and RSU-origin BSHH vectors by preventing any single node from forging a signature under another node’s identity without breaking the underlying lattice assumption.

Location-Binding Signatures for ME Mitigation. As established in Section 3.4.6, the ME attack is a false witness attestation problem that cannot be addressed by content-authentication alone. Each topology observation report is therefore constructed by binding the observed link to the reporter’s GPS-attested position and RSSI measurement before signing:

$$m'_{V_k} = e_{ij} \parallel \text{pos}_{V_k} \parallel \text{RSSI}_{V_k \leftarrow V_i} \parallel \tau_s \parallel \text{nonce} \quad (3.63)$$

$$\sigma_{V_k} = \text{Sign}(SK_{V_k}, m'_{V_k}) \quad (3.64)$$

where SK_{V_k} is the private signing key of V_k established via the Kyber+Saber Hybrid-KEM [?], pos_{V_k} is the GPS-attested position of V_k at time τ_s , and $\text{RSSI}_{V_k \leftarrow V_i}$ is the received signal strength from V_i measured at V_k . The controller accepts a single observation report from V_k for link e_{ij} if and only if the signature is valid and both physical consistency conditions are jointly satisfied:

$$\text{Accept}_{V_k}(e_{ij}) \iff \{ \text{Verify}(\sigma_{V_k}, \text{PK}_{V_k}) = 1 \mid \text{pos}_{V_k}, e_{ij} \} \leq r_{\text{comm}} \text{RSSI}_{V_k \leftarrow V_i} \geq \text{RSSI}_{\text{min}}(r_{\text{comm}}) \quad (3.65)$$

To resist collusion among multiple distant attackers, link e_{ij} is inserted into the controller topology database only when a quorum of at least t physically-consistent independent witnesses confirm it:

$$\text{Accept}(e_{ij}) \iff |\{V_k : \text{Accept}_{V_k}(e_{ij}) = 1\}| \geq t \quad (3.66)$$

where $\text{Accept}_{V_k}(e_{ij})$ is defined in eq:me_acceptingleandtisthesamethresholdasin eq:thresholdsi, consistencychecking, eliminatingvehicle-originandRSU-originMEvectorsatthepre-detectionfilterlayer.

Smart Contract Automated Mitigation. Upon a confirmed detection event from either the LW or FS detector, the TemporalEchoMitigator smart contract deployed on Hyperledger Fabric is triggered. The contract performs four

actions atomically. First, the attack metadata is logged to the immutable distributed ledger, providing a tamper-evident audit trail that enables forensic correlation of low-rate attack events across observation windows. Second, the variant-specific mitigation action is executed: for TTW and BSHH, an OpenFlow FlowMod DROP rule is pushed to the SDN controller to isolate the offending node, and the associated session key is revoked through the KEM module; for ME, the falsely inferred paths are invalidated in the controller topology database and a reroute FlowMod is pushed to restore correct forwarding. Third, the offending node is flagged for mandatory re-authentication before re-admission. Fourth, an AttackDetected event is emitted to the Application and Analytics Layer for operator notification. Blockchain serves a dual role in this framework: as a *forensic detection enabler* through immutable temporal event correlation, enabling detection of low-rate attacks whose individual events fall below the detection threshold but whose cumulative pattern does not; and as an *automated mitigation executor* that enforces isolation and routing correction without human intervention. This dual active role distinguishes the proposed framework from all prior SDVN blockchain deployments, which use blockchain exclusively for passive audit [? ?]. The complete mitigation procedure is given in Algorithm 4.

Alert Interface. The alert object passed from either the LW or FS detector to the TemporalEchoMitigator smart contract is defined as the tuple:

$$\mathcal{O} = (v_{\text{id}}, \alpha, \hat{y}_v, \mathcal{S}_{\text{trig}}, t_{\text{alert}}) \quad (3.67)$$

where v_{id} is the offending node identifier, $\alpha \in \{\text{TTW}, \text{BSHH}, \text{ME}\}$ is the detected attack variant, $\hat{y}_v$ is the anomaly score from $\text{eq:anomaly_score}(\text{ortheweightedsignaturescores}(e)f)$, $\{1, \dots, 9\}$ is the set of triggered signature indices, and t_{alert} is the detection timestamp. The Hyperledger Fabric chaincode receives $\mathcal{O}$ via the Northbound API of the SDN controller and executes the mitigation logic under PBFT consensus among the consortium RSU nodes.

Dual-Path Conflict Resolution. When both the LW and FS detection paths raise alerts for the same node v simultaneously, the smart contract resolves the conflict by taking the union of detected variants and applying the most restrictive mitigation action. Specifically, node isolation (FlowMod DROP) takes precedence over path invalidation, ensuring that a node flagged for TTW or BSHH by the LW path is isolated even if the FS path simultaneously detects ME, preventing a race condition between DROP and reroute FlowMod rules at the SDN controller.

Consensus and Execution Latency. The TemporalEchoMitigator contract executes under Practical Byzantine Fault Tolerant (PBFT) consensus among the Hyperledger Fabric consortium nodes, which correspond to the RSUs in the SDVN deployment. PBFT tolerates up to $\lfloor (n-1)/3 \rfloor$ Byzantine RSU nodes, where n is the total number of consortium participants. Smart contract execution latency on Hyperledger Fabric is typically 50–200 ms [?], which combined with the TGN inference latency and FlowMod propagation time must remain within the 100 ms safety latency budget. This constraint motivates the parallel deployment of the LW path, which responds within $O(|\mathcal{W}|)$ time without blockchain overhead, providing immediate provisional isolation while the FS path completes its full verification

and logging cycle.

Mobility-Aware Mitigation Timing. The 100 ms safety latency constraint must be interpreted in the context of the SDVN mobility model. At highway speeds, the minimum link lifetime is approximately 9 s eq:linklifetime, which corresponds to 90 beacon intervals. A 100 ms latency, provides confirmed isolation before the second beacon interval elapses, ensuring that even critical services such as collision avoidance. For ME attacks, which require the false topology to be detected mitigation less critical than for TTW and BSHH.

Algorithm code for Lightweight rule-based signature detection

aa

Algorithm code for Lightweight HMAC-Timestamp-Nonce Mitigation

Algorithm code for Full-Stack TGN Detection

Algorithm code for Full-Stack Smart Contract Mitigation

11

3.5 Attack Impact Framework

Detection Metrics (Conceptual)

Table 3.1: Detection Metrics [19]

Metric	Description
Precision	Ratio of correctly detected attacks to the total number of detected attacks
Recall	Ratio of correctly detected attacks to the total number of actual attacks
Matthews Correlation Coefficient (MCC)	Correlation-based performance measure that considers true positives, true negatives, false positives, and false negatives, and is robust to class imbalance
Detection Latency	Time elapsed between the occurrence of an attack and its detection by the system
Computational Complexity	Time and space complexity of the detection algorithm with respect to the input size

3.6 Cross-Domain Temporal Pattern Analysis

Temporal deception patterns show strong structural similarity between SDVN and financial fraud [1, 3]:

- Time reordering $\leftrightarrow$ transaction replay [4]
- Liveness spoofing $\leftrightarrow$ account heartbeat fraud [11]
- Phantom duplication $\leftrightarrow$ shell-company networks [12]

Graph-theoretic analysis shows isomorphic anomaly structures in temporal adjacency matrices [1, 4].

Algorithm 1 Lightweight Rule-Based Signature Detection (LW-DETECT).

```

1: procedure LW-DETECT( $e, \mathcal{W}, w, \theta_{\text{LW}}$ )
2:   APPEND( $\mathcal{W}, e$ )
3:   if  $|\mathcal{W}| > W_{\text{max}}$  then
4:     REMOVE_OLDEST( $\mathcal{W}$ )
5:   end if
6:    $s \leftarrow 0$ 
7:   for  $k \leftarrow 1$  to 9 do
8:      $f_k \leftarrow \text{EVAL\_SIG}(k, e, \mathcal{W})$ 
9:      $s \leftarrow s + w_k \cdot f_k$ 
10:  end for
11:  if  $s > \theta_{\text{LW}}$  then
12:     $\alpha \leftarrow \text{CLASSIFY\_ATTACK}(s, e)$ 
13:    return ALERT( $\alpha, e, s$ )
14:  end if
15:  return BENIGN
16: end procedure

17: function EVAL_SIG( $k, e, \mathcal{W}$ )
18:   if  $k = 1$  then
19:     return  $\mathbf{1}[\tau_r^{(e)} - \tau_s^{(e)} > T_b + \epsilon]$  ▷ TTW-S1: Eq. eq:ttw_s1
20:      $k = 2$ 
21:     return  $\mathbf{1}[\exists j \in \mathcal{W} : \tau_s^{(j)} > \tau_s^{(e)} \wedge \tau_r^{(j)} < \tau_r^{(e)}]$  ▷ TTW-S2: Eq. eq:ttw_s2
22:      $k = 3$ 
23:     return  $\mathbf{1}[\exists j \in \mathcal{W} : |\tau_s^{(j)}(e_{ij}) - \tau_s^{(e)}(e_{ij})| > T_b]$  ▷ TTW-S3:
Eq. eq:ttw_s3
24:      $k = 4$ 
25:     return  $\mathbf{1}[\exists V_a \neq V_b : \text{Sender}(H^{V_a}) = \text{Sender}(H^{V_b})]$  ▷ BSHH-S1:
Eq. eq:bshh_s1
26:      $k = 5$ 
27:     return  $\mathbf{1}[\tau_s^{(e)}(V_i) < \tau_s^{\text{prev}}(V_i)]$  ▷ BSHH-S2: Eq. eq:bshh_s2
28:      $k = 6$ 
29:     return  $\mathbf{1}[\text{HB}(V_i, t) \wedge \text{Beacon}(V_i, [t - W, t])]$  ▷ BSHH-S3:
Eq. eq:bshh_s3
30:      $k = 7$ 
31:     return  $\mathbf{1}[|\mathcal{R}(e_{ij}, t)| > \rho_{\text{max}}(\lambda, r_{\text{comm}})]$  ▷ ME-S1:
Eq. eq:me_s1
32:      $k = 8$ 
33:     return  $\mathbf{1}[|\mathcal{P}(V_i, V_j)|_t - |\mathcal{P}(V_i, V_j)|_{t-1} > \delta_{\text{max}}]$  ▷
ME-S2: Eq. eq:me_s2
34:      $k = 9$ 
35:     return  $\mathbf{1}[d(\text{pos}_{V_k}, e_{ij}) > r_{\text{comm}} \vee \text{RSSI}_{V_k} < \text{RSSI}_{\text{min}}]$ 
▷ ME-S3: Eq. eq:me_s3
36:
37:   end function

38:   function CLASSIFY_ATTACK( $s, e$ )
39:      $\mathcal{T} \leftarrow \{k : f_k = 1, k \in \{1, 2, 3\}\}$ 
40:      $\mathcal{B} \leftarrow \{k : f_k = 1, k \in \{4, 5, 6\}\}$ 
41:      $\mathcal{M} \leftarrow \{k : f_k = 1, k \in \{7, 8, 9\}\}$ 
42:     return  $\arg \max(|\mathcal{T}|, |\mathcal{B}|, |\mathcal{M}|)$ 
43:   end function

```

Algorithm 2 Lightweight HMAC-Timestamp-Nonce Mitigation (LW-MITIGATE).

```
1: procedure LW-MITIGATE( $m, \tau_s, \tau_r, \text{nonce}, K_{V_i}, \mathcal{N}_{\text{seen}}$ )
2:    $m' \leftarrow m \parallel \tau_s \parallel \text{nonce}$   $\triangleright$  Eq. eq:hmacconstruction
3:    $\text{MAC}_{\text{exp}} \leftarrow \text{HMAC-SHA256}(K_{V_i}, m')$   $\triangleright$  Eq. eq:hmaccomputation
4:    $\text{MAC}_{\text{exp}} \neq \text{MAC}_{\text{received}}$   $\triangleright$  Eq. eq:hmacverify
5:   return DROP(invalid MAC)
6:
7:   if  $|\tau_r - \tau_s| > T_b + \epsilon$  then  $\triangleright$  Eq. eq:freshness
8:     return DROP(stale message)
9:   end if
10:  if  $\text{nonce} \in \mathcal{N}_{\text{seen}}$  then  $\triangleright$  Eq. eq:nonce
11:    return DROP(replayed nonce)
12:  end if
13:  ADD( $\mathcal{N}_{\text{seen}}, \text{nonce}$ )
14:  return ACCEPT( $m$ )
15: end procedure
```

Algorithm 3 Full-Stack Temporal GNN Detection (FS-DETECT).

```
1: procedure FS-DETECT( $\mathcal{G}_t, \Theta, \theta_{\text{FS}}$ )
2:    $\mathbf{X}_t \leftarrow \text{EXTRACT\_FEATURES}(\mathcal{G}_t)$   $\triangleright$  Eq. eq:nodefeatures
3:   each  $v \in \mathcal{V}_t$ 
4:      $\Delta t_v \leftarrow t - t_v^{\text{last}}$ 
5:      $\phi_v \leftarrow \log(1 + \Delta t_v / T_b)$ 
6:      $\mathbf{m}_v(t) \leftarrow \text{GRU}(\mathbf{h}_v(t^-), \mathbf{x}_v(t), \phi_v)$   $\triangleright$  Eq. eq:tgnmemory
7:      $\mathbf{h}_v^{(0)} \leftarrow \mathbf{m}_v(t)$ 
8:
9:     for  $l \leftarrow 0$  to  $L - 1$  do
10:      for each  $v \in \mathcal{V}_t$  do
11:         $\mathbf{h}_v^{(l+1)} \leftarrow \sigma(\mathbf{W}^{(l)} \cdot \text{MEAN}(\{\mathbf{h}_u^{(l)} \odot \mathbf{A}_{uv}(t) : u \in \mathcal{N}(v, t)\}) + \mathbf{b}^{(l)})$ 
12:       $\triangleright$  Eq. eq:tgnmessage
13:    end for
14:     $\mathcal{A} \leftarrow \emptyset$ 
15:    for each  $v \in \mathcal{V}_t$  do
16:       $\hat{y}_v \leftarrow \sigma(\mathbf{w}^\top \mathbf{h}_v^{(L)})$   $\triangleright$  Eq. eq:anomalyscore
17:       $\hat{y}_v > \theta_{\text{FS}}$ 
18:       $\mathcal{A} \leftarrow \mathcal{A} \cup \{(v, \hat{y}_v)\}$ 
19:    end for
20:    return  $\mathcal{A}$ 
21:  end procedure
22:
23:  function EXTRACT_FEATURES( $\mathcal{G}_t$ )
24:    for each  $v \in \mathcal{V}_t$  do
25:       $\mathbf{x}_v \leftarrow [\text{id}_v, \tau_s^{(v)}, c_v^W, \Delta s_v, \rho_v]$   $\triangleright$  Eq. eq:nodefeatures
26:       $t_v^{\text{last}} \leftarrow \tau_r^{(v)}$ 
27:    end for
28:    return  $\mathbf{X}_t$ 
29:  end function
```

Algorithm 4 Full-Stack Smart Contract Mitigation (FS-MITIGATE).

```
1: procedure FS-MITIGATE( $\mathcal{A}, \sigma, \mathcal{L}, \theta_{\text{FS}}, t$ )
2:   for each  $(v, \hat{y}_v) \in \mathcal{A}$  do
3:     LOG( $\mathcal{L}, \{v, \hat{y}_v, t\}$ ) ▷ Immutable ledger entry
4:      $\alpha \leftarrow \text{IDENTIFY\_VARIANT}(v, \mathcal{A})$ 
5:     if  $\alpha \in \{\text{TTW}, \text{BSHH}\}$  then
6:       VERIFY_THRESHOLD_SIG( $\sigma_v$ ) ▷ Eq. eq:thresholdsig
7:       PUSH_FLOWMOD(DROP,  $v$ ) ▷ SDN controller isolation
8:       REVOKE_SESSION_KEY( $v$ ) ▷ Kyber+Saber KEM revocation
9:     else if  $\alpha = \text{ME}$  then
10:       $\mathcal{W}_v \leftarrow \text{GET\_WITNESSES}(e_{ij}, t)$ 
11:      VERIFY_QUORUM( $\mathcal{W}_v, t$ ) ▷ Eq. eq:mequorum
12:      INVALIDATE_PATHS( $\mathcal{P}_v^{\text{false}}$ )
13:      PUSH_REROUTE_FLOWMOD() ▷ SDN routing correction
14:    end if
15:    FLAG_REAUTH( $v$ ) ▷ Block re-admission until re-authenticated
16:    EMIT(ATTACKDETECTED,  $\{v, \alpha, \hat{y}_v, t\}$ )
17:  end for
18: end procedure

19: function VERIFY_THRESHOLD_SIG( $\sigma_v$ )
20:    $c \leftarrow 0$ 
21:   for each  $(\sigma_i, \text{msg}_i, \text{PK}_{V_i}) \in \sigma_v$  do
22:     if Verify( $\sigma_i, \text{msg}_i, \text{PK}_{V_i}$ ) = 1 then
23:        $c \leftarrow c + 1$ 
24:     end if
25:   end for
26:   if  $c < t$  then ▷ Eq. eq:thresholdsig
27:     if thenreturn FAIL
28:   end if
29:   return PASS
30:

31: function VERIFY_QUORUM( $\mathcal{W}_v, t$ )
32:    $q \leftarrow 0$ 
33:   for each  $V_k \in \mathcal{W}_v$  do
34:     if Verify( $\sigma_{V_k}, \text{PK}_{V_k}$ ) = 1 then
35:       if  $d(\text{pos}_{V_k}, e_{ij}) \leq r_{\text{comm}}$  then ▷ Eq. eq:meacceptsingle
36:         if  $\text{RSSI}_{V_k} \geq \text{RSSI}_{\text{min}}$  then
37:            $q \leftarrow q + 1$ 
38:         end if
39:       end if
40:     end if
41:   end for
42:   if  $q < t$  then ▷ Eq. eq:mequorum
43:     if thenreturn FAIL
44:   end if
45:   return PASS
46:
```

Chapter 4

Timeline and Resource Required

...

4.0.1 Baseline Selection and Benchmarking Rationale

External Baselines

...

4.0.2 Baseline Selection and Benchmarking Rationale

External Baselines

- **B1** — **Deng et al.** [?]: Threshold-based countermeasures against topology poisoning in SDVNs using beacon-based discovery [?]. Closest existing work; same vehicular mobility context. Limitation: fixed thresholds, no temporal modeling, no mobility adaptation. *Note: Students must verify from IEEE Xplore (DOI: 10.1109/TITS.2024.3432087) whether discovery is purely beacon-based or mixed with LLDP for RSU–RSU links.*
- **B2** — **Mekonen et al.** [?]: RSU-level ensemble learning (KNN with bagging) detecting position falsification attacks via sequential BSM analysis [?]. Selected because it uses beacon messages directly at the RSU, and position falsification via BSMS is the closest existing beacon-based attack to the ME false witness attestation mechanism. Limitation: detects position-payload anomalies only; cannot detect timestamp ordering violations (TTW) or heartbeat identity spoofing (BSHH).
- **B3** — **TopoSleuth** [?]: Multi-layered topology poisoning defense with decoy links, behavioral profiling, and multi-hop validation [?]. Most recent state-of-the-art topology defense; LLDP trigger conditions

adapted to equivalent beacon timestamp and reporter-count conditions for the simulation. Limitation: no vehicular mobility modeling, no cryptographic mitigation.

Internal Ablation Baselines

- **A1 (LW-only):** Rule-based signatures and HMAC; no TGN or blockchain.
- **A2 (FS-Det-only):** TGN detection without smart contract mitigation.
- **A3 (Static-GCN):** TGN replaced with static GCN; no temporal memory or time-elapsd encoding.
- **A4 (No-Mobility):** Fixed $\rho_{\max}$ and fixed window W ; no mobility-aware signature calibration.
- **A5 (No-LBS):** Full-stack without location-binding signatures for ME.
- **A6 (No-Crypto-Filter):** TGN without pre-detection HMAC filter.

Performance Metrics

Detection:

- **MCC** — primary detection metric; robust to class imbalance eq:mcc.
- **AUROC** — threshold-independent detection capability.
- **Detection Latency** (T_{det} , ms) — time from attack injection to alert; compared against $T_b = 100$ ms.

Routing performance impact:

- **Packet Delivery Ratio (PDR)** — fraction of packets successfully delivered under attack and post-mitigation.
- **End-to-End Latency** (T_{e2e} , ms) — average packet delivery time under attack and post-mitigation.

All metrics are reported as mean $\pm$ standard deviation over five independent NS-3/SUMO simulation runs.

4.1 Timeline

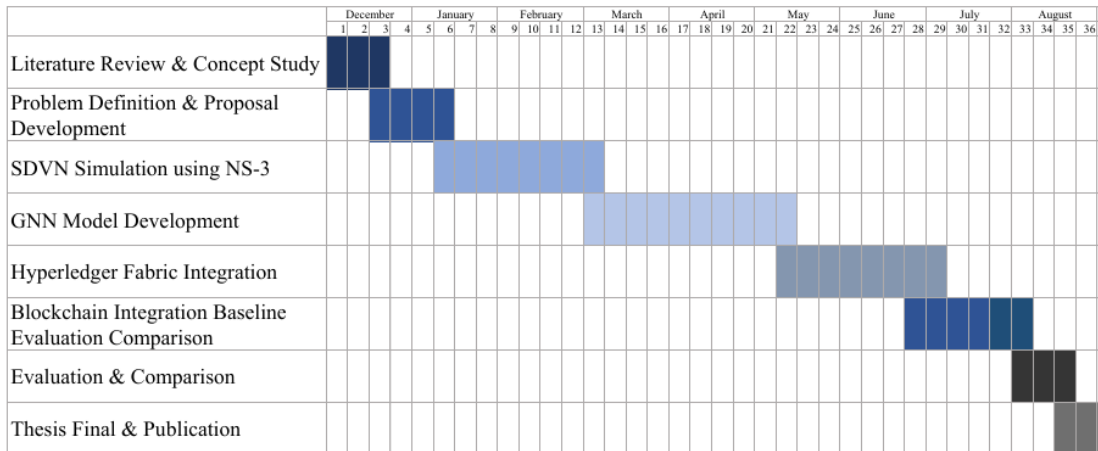


Figure 4.1: Project Timeline (Gantt Chart)

4.2 Resource Required

4.2.1 Hardware Resources

Table 4.1: Hardware Resource Requirements

Resource	Specification	Quantity	Source	Cost
GPU Server	NVIDIA A100 (40GB VRAM)	1	Uni. Provided	0
CPU Server	Intel Xeon (32 cores)	1	Uni. Provided	0
RAM	256GB DDR4	1	Uni. Provided	0
Storage	2TB NVMe SSD	1	Uni. Provided	0
Subtotal Hardware				\$0

4.2.2 Software Resources (All Open-Source/Free)

Table 4.2: Software Resource Requirements

Software	Purpose	Cost
PyTorch Geometric	GNN implementation	Free
TensorFlow + Spektral	Alternative GNN framework	Free
Hyperledger Fabric	Blockchain framework	Free
NS-3	Network simulator	Free
SUMO	Vehicular mobility simulator	Free
Go / JavaScript	Smart contract development	Free
Python	Data processing and analysis	Free
Ubuntu 24.04.3 LTS	Operating system	Free
Subtotal Software		\$0

4.2.3 Budget Summary

Table 4.3: Overall Project Budget

Category	Cost
Hardware	\$0
Software	\$0
Personnel (optional)	Variable
Cloud Services (optional)	\$0-\$1,000/month
Minimum Total	\$0
Recommended Total	\$0-\$1000

Chapter 5

Conclusion

Software-Defined Vehicular Networks (SDVNs) enable flexible and programmable vehicular communication, but they also introduce new security risks in their control plane. In particular, this project focuses on identifying and understanding a class of time-based threats, referred to as Temporal-Echo Topology Attacks, which exploit the ordering and timing of topology control messages rather than creating obviously false network structures.

Three attack variants—Topology Time-Warp (TTW), Beacon State Heartbeat Hijack (BSHH), and Multipath Echo (ME)—have been identified as potential threats that can mislead the SDN controller while preserving the apparent legitimacy of control packets. These attacks highlight limitations in existing topology poisoning defenses, which mainly rely on structural validation or rule-based detection and may be ineffective against temporally manipulated but syntactically valid messages.

At the current stage, this work is limited to studying and characterizing these attack behaviors. The next phases of the project will focus on exploring suitable detection and verification approaches and evaluating modern techniques, such as graph-based learning and distributed trust mechanisms, as possible solution directions. This proposal therefore establishes a clear problem definition and research motivation, which will guide the design and implementation of effective solutions in later stages of the project.

References

- [1] G. Yang, F. Wang, and K. Yeh, “GNN-enhanced traffic anomaly detection for next-generation SDN-enabled consumer electronics,” *arXiv preprint arXiv:2510.07109*, 2024.
- [2] I. M. Varma and N. Kumar, “A comprehensive survey on SDN and blockchain-based secure vehicular networks,” *Vehicular Communications*, vol. 44, p. 100663, 2023.
- [3] R. Renuga Devi, J. Emerson Raja, and Y. Boon Chin, “Reinforcement learning with graph neural network (RL-GNN) fusion for real-time financial fraud detection: a context-aware community mining approach,” *Scientific Reports*, vol. 15, p. 42953, 2025.
- [4] D. Cheng, Y. Zou, S. Xiang, and C. Jiang, “Graph Neural Networks for Financial Fraud Detection: A Review,” *arXiv preprint arXiv:2411.05815*, 2024.
- [5] S. B. Veeram et al., “Design of an integrated model with temporal graph attention and transformer-augmented RNNs for enhanced anomaly detection,” *Scientific Reports*, vol. 15, p. 2692, 2025.
- [6] Y. Kim et al., “Temporal Graph Networks for Graph Anomaly Detection in Financial Networks,” *arXiv preprint arXiv:2404.00060*, 2024.
- [7] N. Zhao, H. Wu, and X. Zhao, “Consortium Blockchain-Based Secure Software Defined Vehicular Network,” *Mobile Networks and Applications*, vol. 25, no. 1, pp. 314–327, 2020.
- [8] S. Choudhary and S. Dorle, “Secured SDN Based Blockchain: An Architecture to Improve the Security of VANET,” *International Journal of Electrical and Computer Engineering Systems*, vol. 13, no. 2, pp. 145–153, 2022.
- [9] R. Alhabib and P. Yadav, “Hyperledger Fabric Platform for Secure and Efficient Data Sharing in Autonomous Vehicles,” in *WinCom 2024*, 2024.
- [10] M. R. Zafar et al., “Ride-Hailing for Autonomous Vehicles: Hyperledger Fabric-Based Secure and Decentralized Blockchain Platform,” in *IEEE International Conference on Big Data*, 2021.

- [11] B. Shi *et al.*, “An edge feature aware heterogeneous graph neural network model to support tax evasion detection,” *Expert Systems with Applications*, vol. 213, Part B, p. 118903, 2023.
- [12] D. Sisodia and D. S. Sisodia, “A transfer learning framework towards identifying behavioral changes of fraudulent publishers in pay-per-click model of online advertising for click fraud detection,” *Expert Systems with Applications*, vol. 232, p. 120922, 2023.
- [13] N. Jayakrishna and N. N. Prasanth, “A hybrid deep learning model for detection and mitigation of DDoS attacks in VANETs,” *Scientific Reports*, vol. 15, p. 34170, 2025.
- [14] L. Xie *et al.*, “Towards a Blockchain and Software-Defined Vehicular Networks Approaches to Secure Vehicular Social Network,” in *IEEE Conference on Communications and Network Security*, 2018.
- [15] J. Kang *et al.*, “Blockchain-Based Distributed Software-Defined Vehicular Networks via Deep Q-Learning,” in *ACM Symposium on Design and Analysis of Intelligent Vehicular Networks and Applications*, 2018.
- [16] X. Shen, V. Jain, and X. Huang, “Detect financial transaction fraud using a Graph Neural Network with Amazon SageMaker,” *AWS Machine Learning Blog*, 2022.
- [17] M. R. Raja *et al.*, “Detecting and preventing money laundering using deep learning and graph analysis,” *International Journal of Advanced Computer Science and Applications*, vol. 16, no. 6, 2025.
- [18] T. Bui, M. Antikainen, and T. Aura, “Analysis of topology poisoning attacks in software-defined networking,” in *Nordic Conference on Secure IT Systems*, 2015.
- [19] L. Jain, V. U., and S. Vollala, “SecTopo: Efficient hybrid model for detecting LLDP topology poisoning attack in programmable data plane,” *Computer Networks*, vol. 238, p. 110121, 2024.
- [20] B. Weintraub, A. Vela Campos, H. Liu, S. R. Chowdhury, and M. K. Reiter, “Exploiting Temporal Vulnerabilities for Unauthorized Access in Intent-Based Networking,” in *Proceedings of ACM Conference on Computer and Communications Security (CCS)*, 2024.
- [21] S. V. Ramani *et al.*, “SDN Framework for Mitigating Time-Based Delay Attack,” *Journal of Information and Communication Technology*, 2022.
- [22] F. Shoaib *et al.*, “Preventing Timing Side-Channel Attacks in Software-Defined Networks,” in *IEEE Network Security and Applications Conference*, 2021.
- [23] S. Liu, S. Shieh, M. K. Reiter, and V. Sekar, “Flow Reconnaissance via Timing Attacks on SDN Switches,” in *Proceedings of IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2017.

- [24] S. Hong, L. Xu, H. Wang, and G. Gu, “Poisoning Network Visibility in Software-Defined Networks: New Attacks and Countermeasures,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2015.
- [25] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, and J. Turner, “OpenFlow: Enabling Innovation in Campus Networks,” *ACM SIGCOMM Review*, 2008.
- [26] U. Kaur *et al.*, “Security Vulnerabilities in VANETs and SDN-based Vehicular Networks,” *International Journal of Computer Networks and Applications*, 2024.
- [27] S. Deng *et al.*, “Poisoning Topology View in Software-Defined Vehicular Networks,” *IEEE Transactions on Intelligent Transportation Systems*, 2024.
- [28] R. F. Fouladi *et al.*, “A DDoS Attack Detection and Defense Scheme Using Time-Series Analysis for SDN,” in *Proceedings of IFIP International Conference on Performance Evaluation and Modeling in Wired and Wireless Networks*, 2020.